

aXet.plugin: Streamline the use of generative AI in our daily services

Use .plugin as a complement in the most used development IDEs.

Currently available for Visual Studio Code, Microsoft VS Professional, Eclipse and IntelliJ.

aXet.plugin – Installation

You can install aXet.plugin in your development IDE as a complement. Currently, it is available for Visual Studio Code, Microsoft VS Professional, Eclipse and IntelliJ. Development for other IDEs is in progress, stay tuned for upcoming releases.

Important: Your user must have access to a project in aXet in order to use the .plugin.

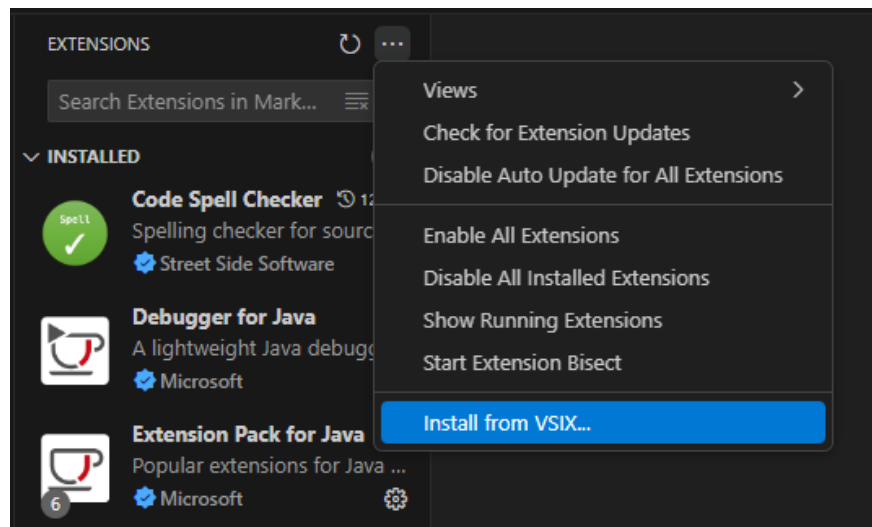
Install aXet.plugin in VS Code

Before starting the procedure, you need to download the **.plugin** installer. Here is a direct [link](#) to the latest version.

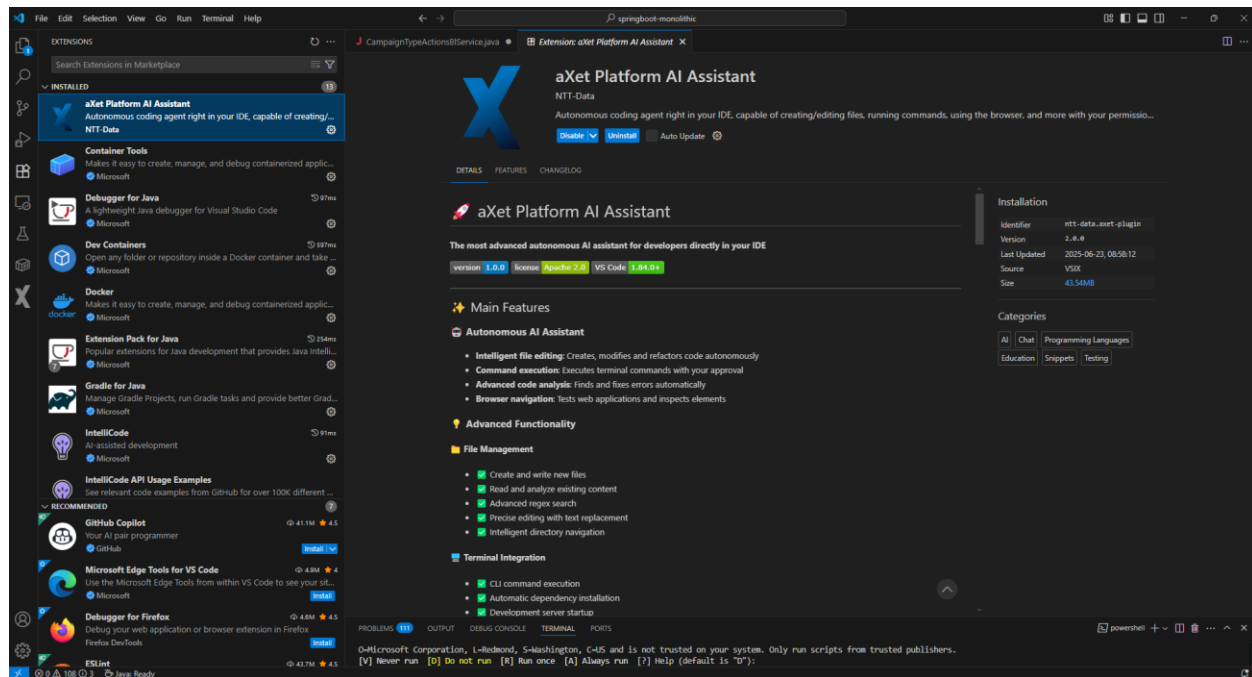
The minimum required version of VS Code to be able to install .plugin is 1.84.0

If you already have the .vsix file, these are the steps to install it in VS Code.

1. Open Visual Studio Code.
2. Click on 'Manage' button and select 'Extensions' from the menu.
3. Click on the three dots to display the context menu and select the option 'Install from VSIX ...'.



4. Select the file you have downloaded and that is the installer for **.plugin**.
5. Wait a few seconds and you will see that a new extension has been installed and is ready to use.



- On 'Details' tab, you can see general information about **aXet.plugin**, such as installation data, main features, use cases, advanced configuration, practical examples...
- On 'Features' tab, you can see information related to the configuration of **aXet.plugin**.

- On 'Changelog' tab, you can see the features and improvements of the published versions of **aXet.plugin**.

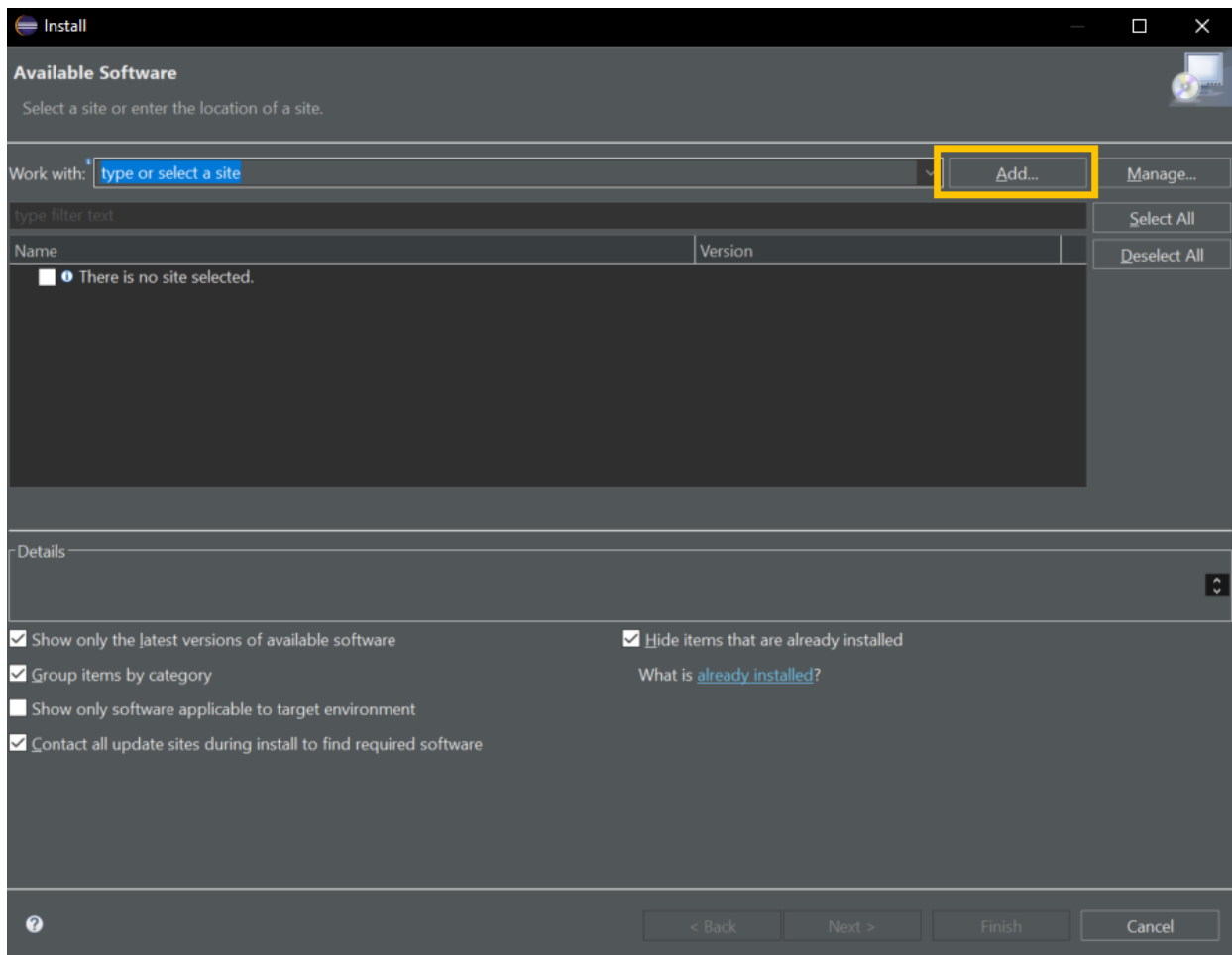
Install aXet.plugin in Eclipse

Before starting the procedure, you need to download the **.plugin** installer. Here is a direct [link](#) to the latest version.

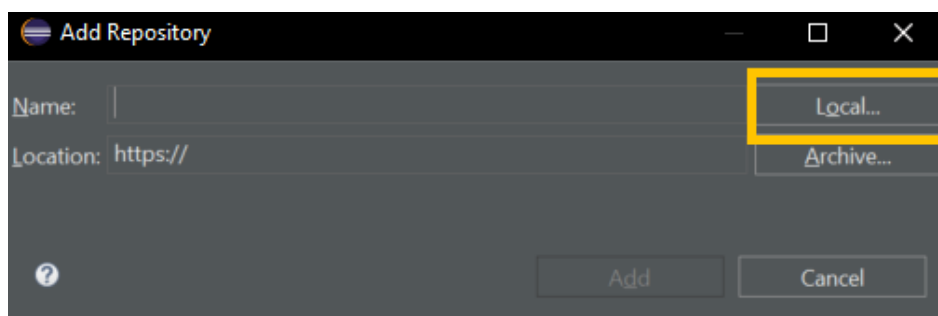
The minimum required version of Eclipse to be able to install .plugin is 2021-03 (4.19)

If you already have the .zip file, these are the steps to install it in Eclipse.

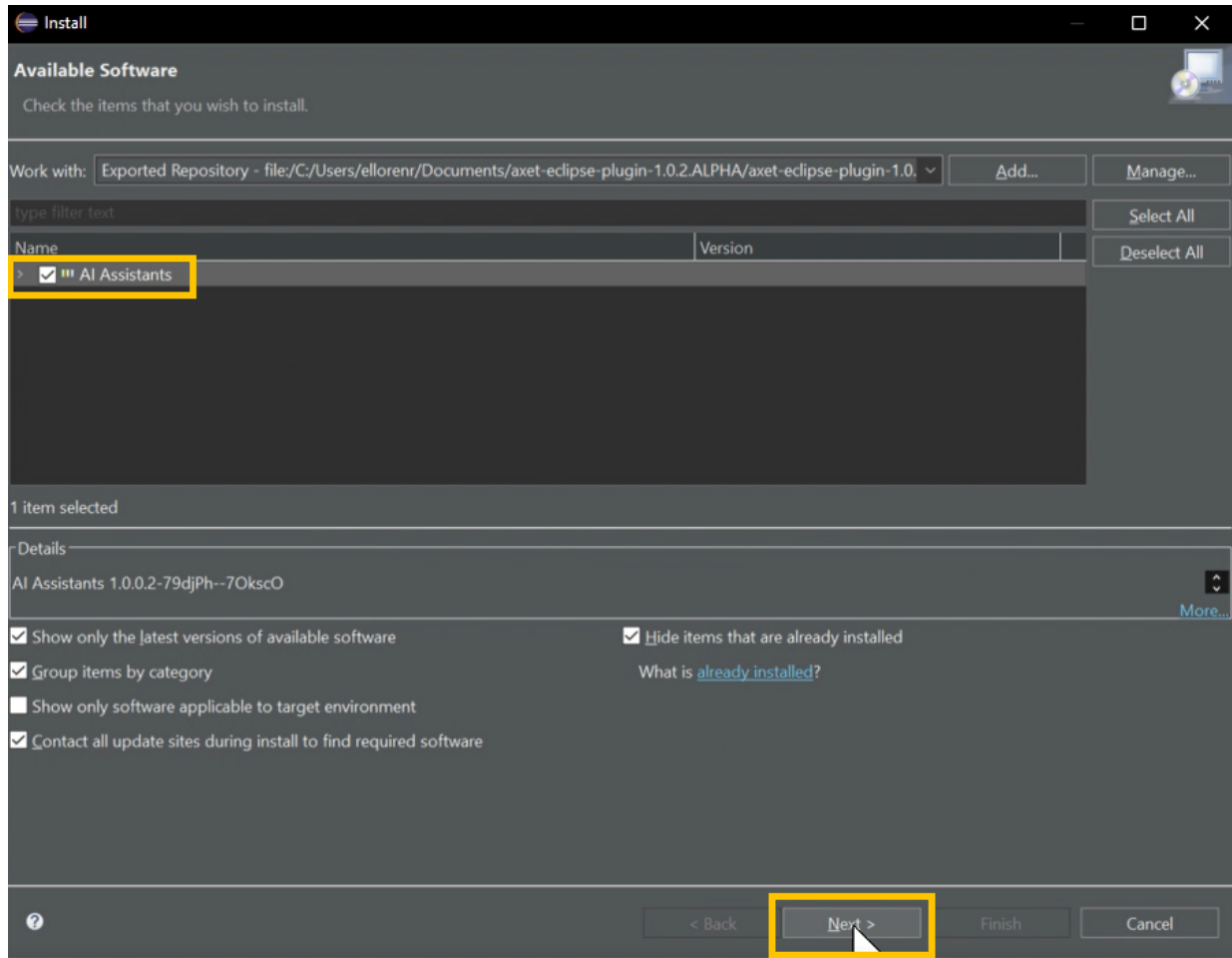
1. Unzip the .zip file.
2. Open Eclipse.
3. Click on 'Help' menu and select 'Install New Software' item.
4. Click on 'Add' button to select the **.plugin** installer.



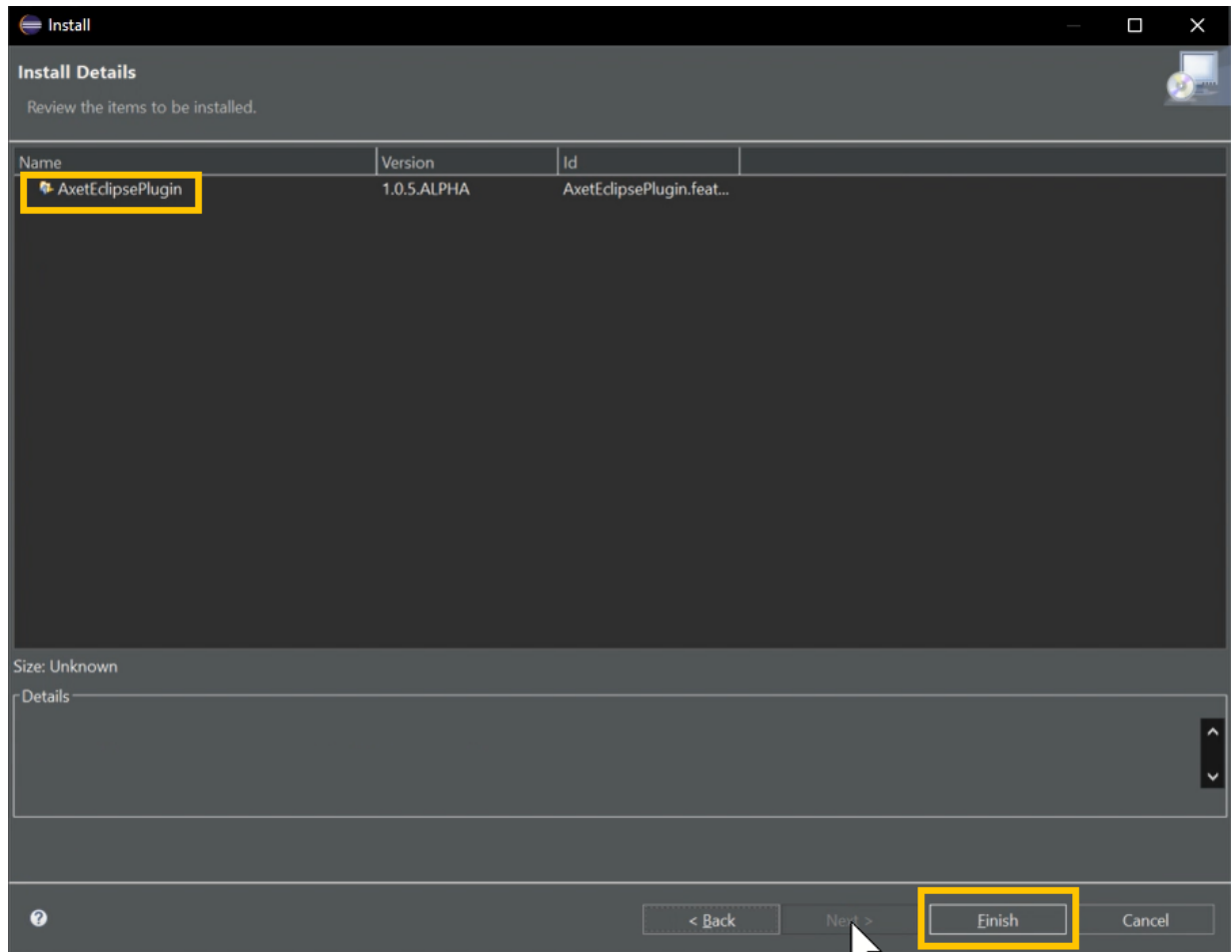
5. In the new window that opens to select the repository where the installer is located, you must select 'Local...'.



6. Select the folder you unzipped in step 1 and click on 'Add' button.
7. Check 'AI Assistants' and click on 'Next' button.



8. Select 'AxetEclipsePlugin' and click on 'Finish' button.



Once **aXet.plugin** is installed, you will get a notification to restart Eclipse to apply the update. Click ‘*Restart Now*’, and just wait for Eclipse to reopen.

Install aXet.plugin in Microsoft VS

Before starting the procedure, you need to download the **.plugin** installer. Here is a direct [link](#) to the latest version.

The minimum required version of Microsoft VS Professional to be able to install .plugin is 2022 17.13.

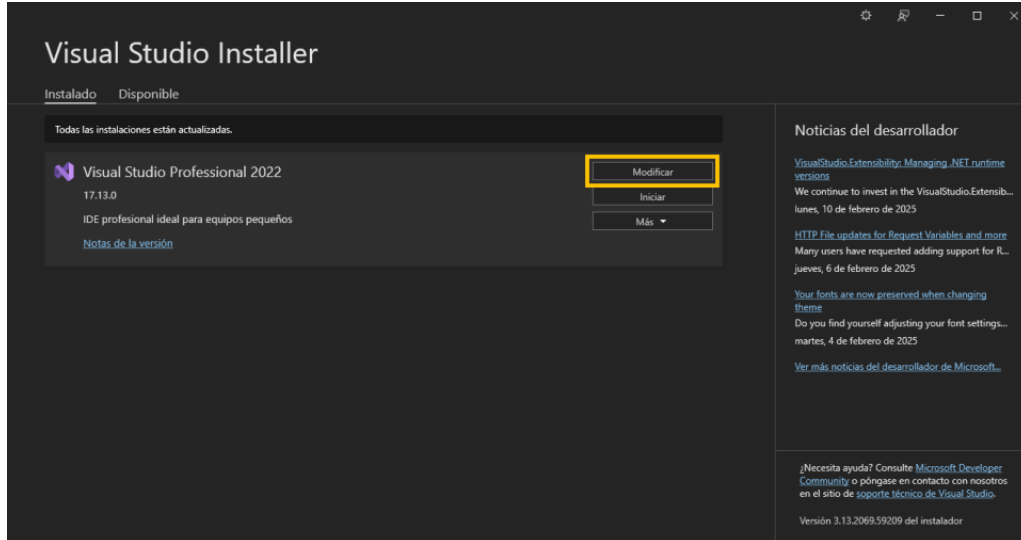
Before installing **aXet.plugin**, you must ensure that the following individual components are installed in Microsoft Visual Studio Professional:

- Visual Studio SDK.
- .NET Framework 4.8.

- .NET SDK.

These are the steps you must follow to install them in case you haven't done so previously.

1. Open Visual Studio Installer and select *'Modify'*.

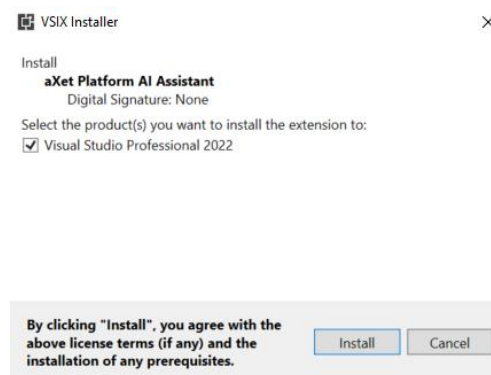


2. Select *'Individual components'* tab and make sure that **Visual Studio SDK**, **.NET Framework 4.8** and **.NET SDK** are checked.
3. Click *'Modify'* if any of the individual components needs to be installed.

You are ready to install **aXet.plugin**.

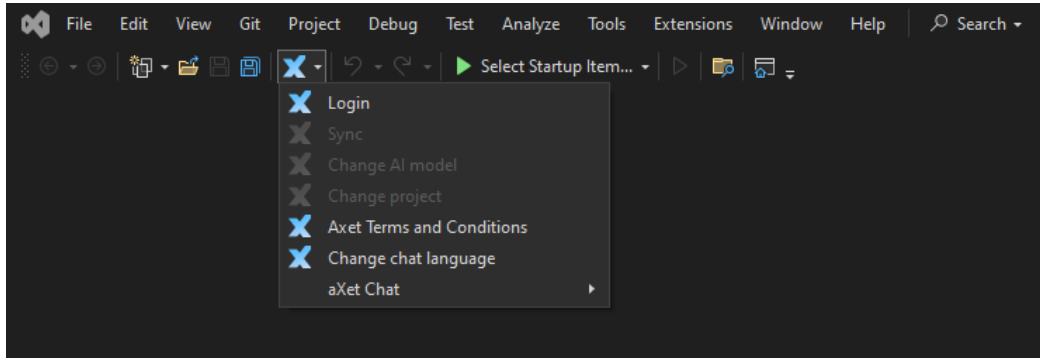
If you already have the .vsix file, these are the steps to install it in Microsoft Visual Studio Professional.

1. Double click on .vsix file and click on *'Install'* button.



2. After updating configuration for Microsoft Visual Studio Professional during installation you must restart the application.

3. You can find **aXet.plugin** available.



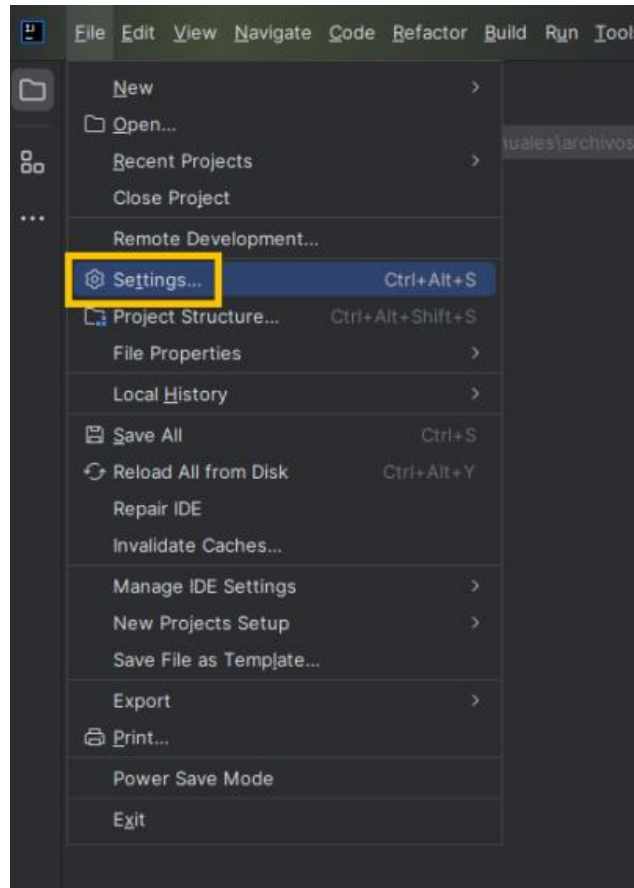
Install aXet.plugin in IntelliJ

Before starting the procedure, you need to download the **.plugin** installer. Here is a direct [link](#) to the latest version.

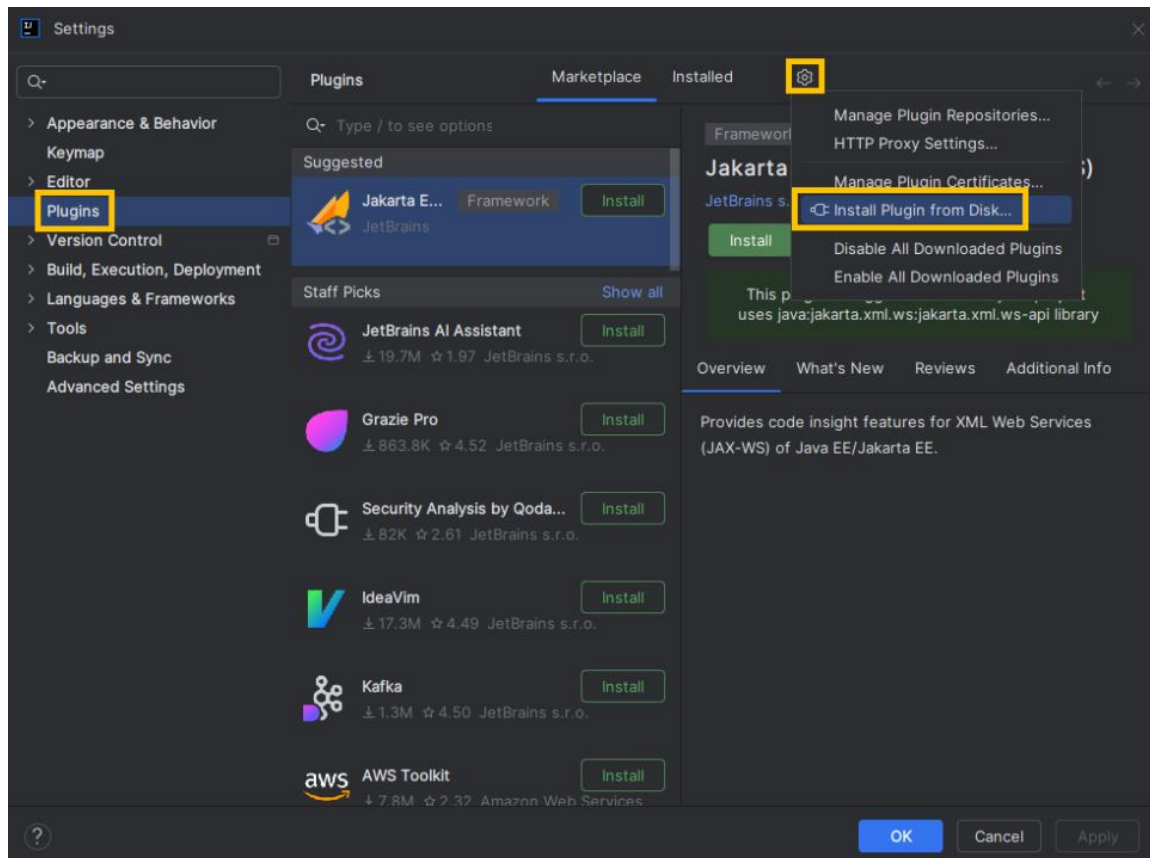
The minimum required version of IntelliJ IDEA to be able to install .plugin is 2023.1.0.

If you already have the .jar file, these are the steps to install it in IntelliJ.

1. Open IntelliJ and the project you're working on.
2. Click on 'File' menu and select 'Settings'.



3. Click on 'Plugins' and in the 'Settings' menu select 'Install Plugin from Disk...'.



4. Select the **.jar** file you have downloaded previously and click on 'Accept' in the window that opens regarding Third-Party plugins.
5. You must restart IntelliJ and you can find **aXet.plugin** available.



aXet.plugin - Using it

You have already installed **.plugin** and are ready to start using it. Select your development IDE to learn more about how to use it.

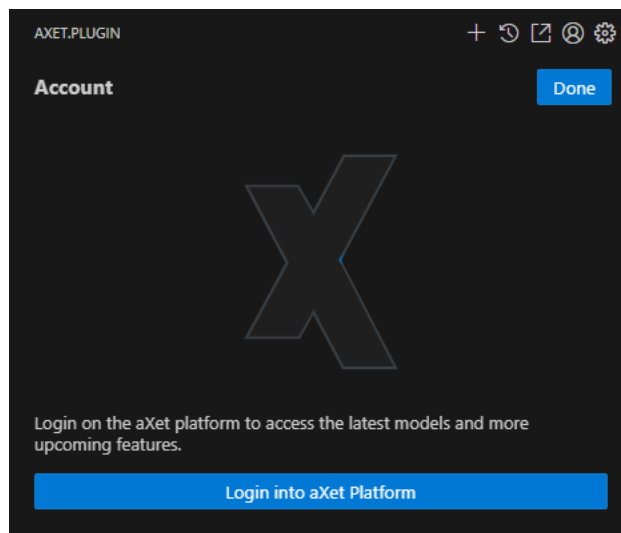
Use aXet.plugin in VS Code

Before starting to use the **plugin** in VS Code, you must log into **aXet**.

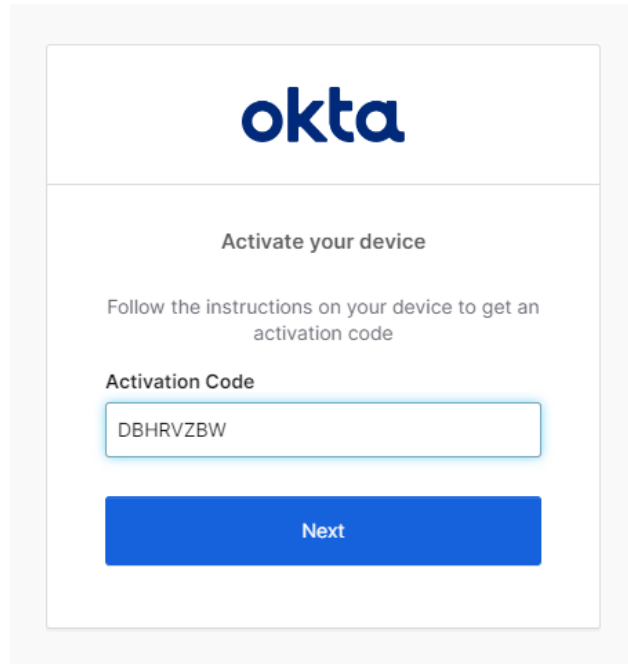
Remember that you must have an active user to use it.

These are the steps you must follow:

1. Click on 'Login into aXet Platform' button in the bottom right corner.



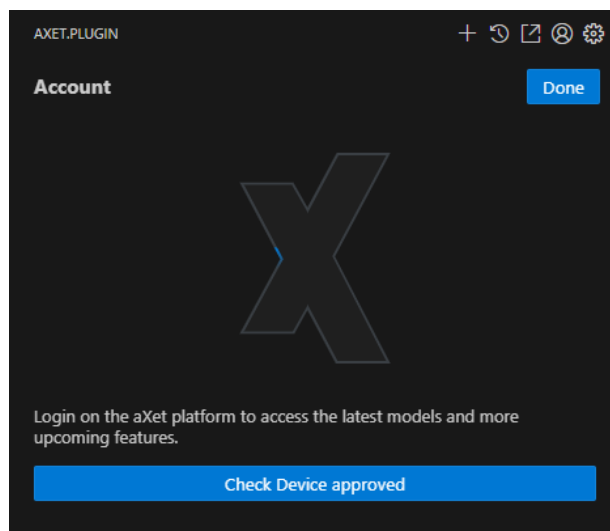
2. A window from Okta opens in the browser with an activation code. Click on the 'Next' button.



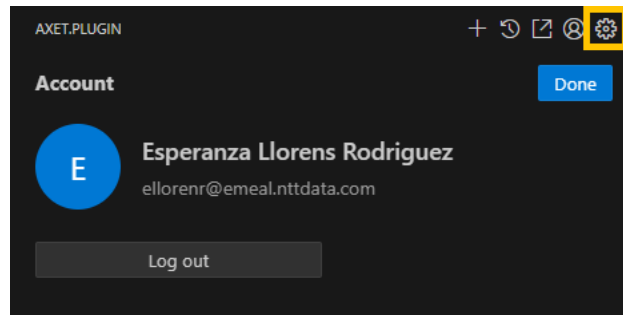
If you are logged into Okta, return to the VS Code interface to continue with the login process.

If you are not logged into Okta, you must follow the steps to do it before continuing with the login in VS Code.

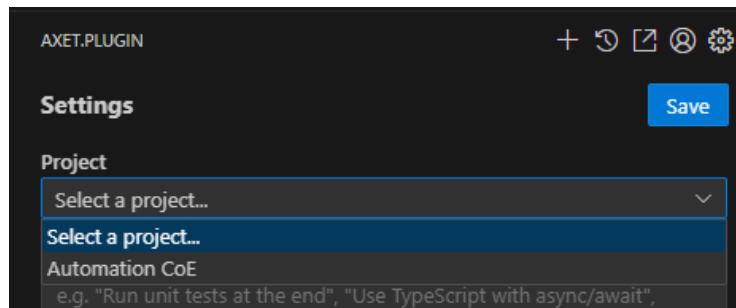
3. Return to VS Code and click on *'Check Device approved'* to go on with login process.



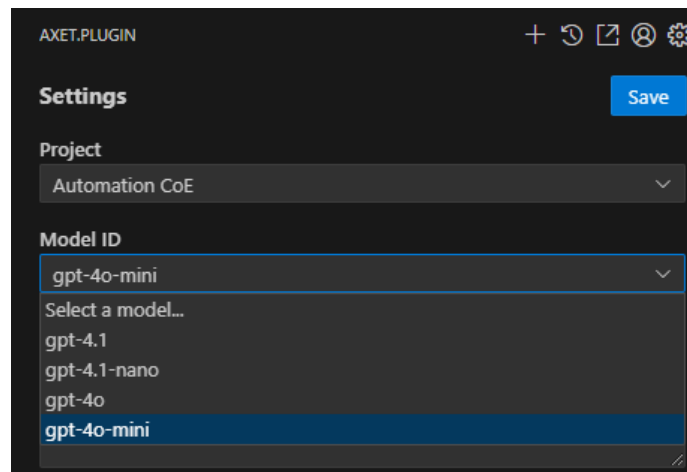
4. You are now logged in and can continue with the configuration of **aXet.plugin**. Click on *'Settings'* button.



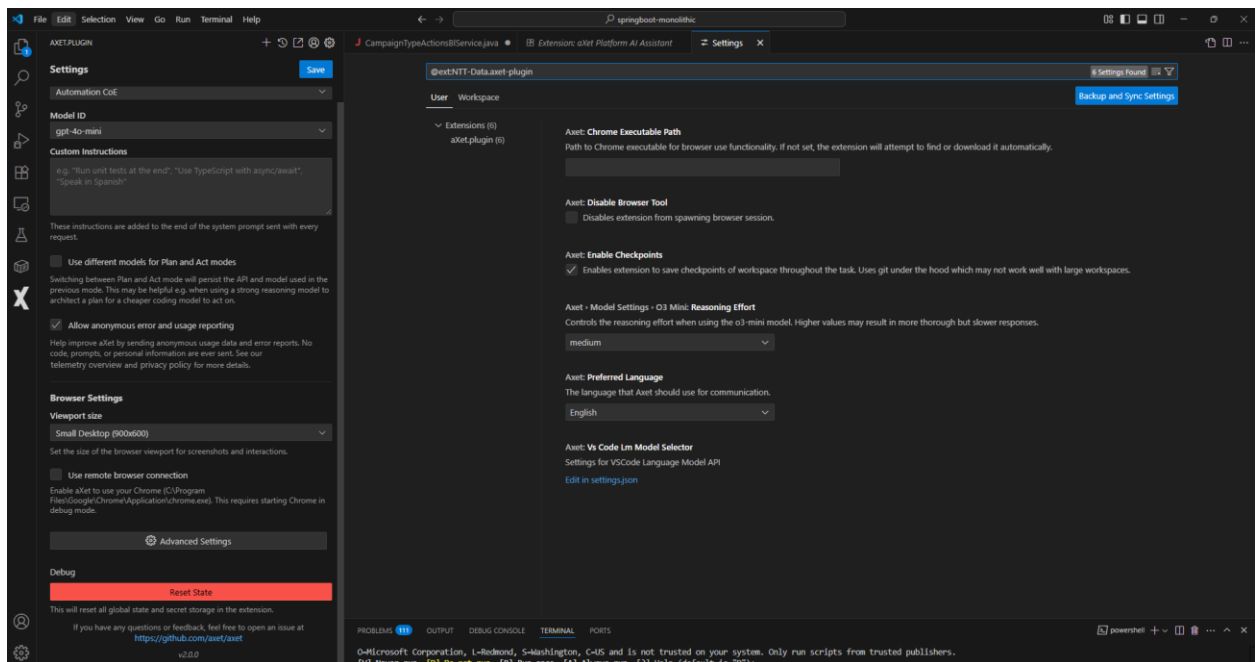
- First, select the project you are going to work on.



- Then, select the default AI model you are going to use. You can change it later.



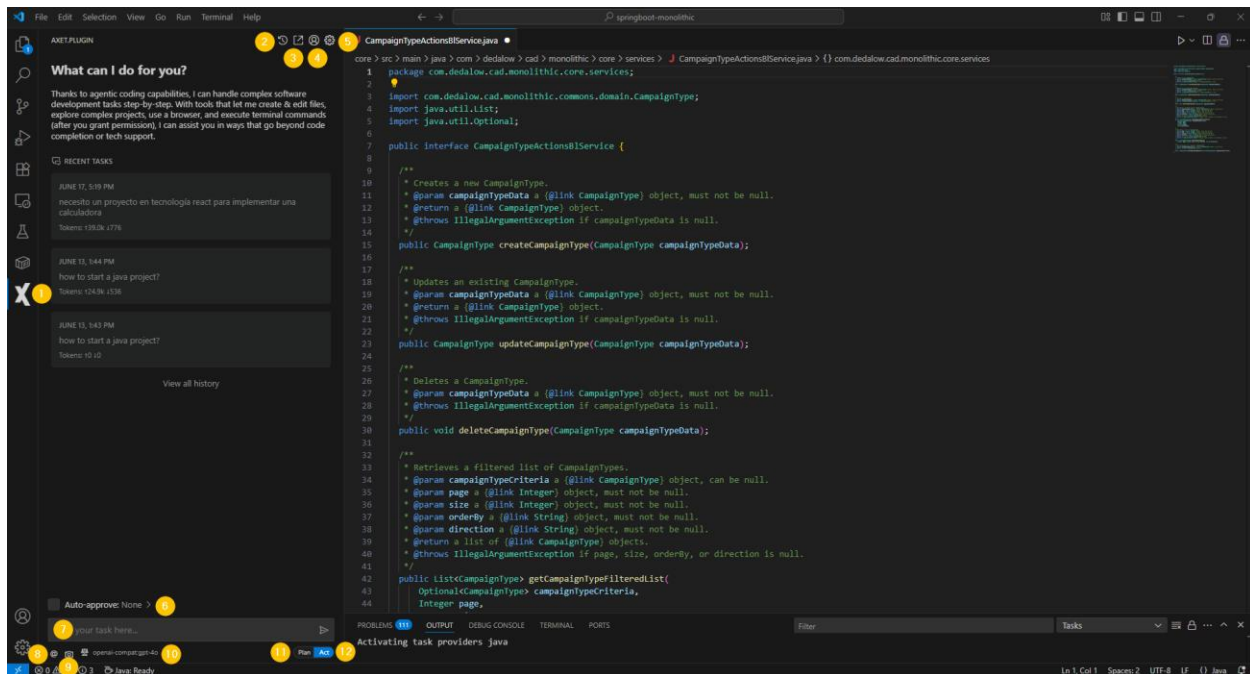
- You can provide additional instructions to your prompts to customize the responses of **.plugin** using the '*Custom Instructions*' parameter.
- You can use different AI models in **plan mode** and **act mode**. In the next section, you will see the difference between these two modes of work of **.plugin**.
- You can setup '*Browser settings*' to select '*Viewport size*' and enable aXet to use Chrome if it is working in debug mode.
- You can also access the '*Advanced Settings*'.



You can setup:

1. **Axet: Chrome Executable Path.** Path to Chrome executable for browser use functionality. If not set, the extension will attempt to find or download it automatically.
2. **Axet: Disable Browser Tool.** Enables/Disables extension from spawning browser session.
3. **Axet: Enable Checkpoints.** Enables/Disables extension to save checkpoints of workspace throughout the task.
4. **Axet > Model Settings > O3 Mini: Reasoning Effort.** Controls the reasoning effort when using the o3-mini model. Higher values may result in more thorough but slower responses. Select from the dropdown: low, medium, high.
5. **Axet: Preferred Language.** The language that aXet should use for communication. Select the language from the dropdown.
6. **Axet: Vs Code Lm Model Selector.** Settings for VSCode Language Model API. Edit in settings.json.

Let's review the VS Code interface when you're ready to work with **.plugin**.



These are the points highlighted in the image:

1. **aXet.plugin shortcut.** When the extension has been installed correctly and you are logged into aXet platform, you can select the shortcut has been created to open the chat and start asking questions to aXet.
2. **aXet.plugin history.** Access the history of queries done. Once in the history list, you can filter, sort, review and delete queries.
3. **Open chat in the editor.**
4. **Access user configuration.** You can logout from here.
5. **aXet.plugin settings.** Access aXet.plugin configuration.
6. **Auto-approve.** Every time .plugin needs to access and/or modify any file in the software project you are working on, it will ask for your permission. You can enable this check and give access to .plugin to all files by selecting the following options: read permission, edit permission, permission to execute commands in the terminal, permission to use the browser. You can also enable notifications from aXet every time it requests access to any file or when a task is completed.
7. **aXet.plugin chat.** Ask an open question and get the answer from aXet.
8. **Add context.** Provide context to your request. This context can be: a URL to fetch contents, Git Commits, some files or some folders from your software development project.

9. **Add images.** Provide context to your request by adding some image files.
10. **Select AI Model / API Provider.** You can change the AI model every time you need it.
11. **Plan mode.** When you request aXet.plugin to execute tasks, select plan mode and it will provide a course of action for you to decide what you want to do next.
12. **Act mode.** When you request aXet.plugin to execute tasks, select act mode and it will automatically makes the necessary changes. It will not only modify or create files but also execute commands in the console, install necessary packages, ...

Use chat in VS Code

Once **aXet.plugin** has been installed and you are logged into **aXet** platform, you can start asking questions to **aXet** using the free chat even if you are not working on a project.

The **aXet.plugin** version for VS Code is an autonomous **agentic** assistant specialized in code generation and performing complex development tasks.

It is capable of planning, reasoning, and executing multi-step actions on its own.

When you request **aXet.plugin** to execute tasks, it has two operating modes:

- Plan mode, in which it provides a course of action for you to decide what you want to do next.
- Act mode, in which it automatically makes the necessary changes. It not only modifies or creates files but also executes commands in the console, installs necessary packages, ...

Just like a real team member, you can see how tasks are executed in real time.

You can ask .plugin to analyze the structure of the project. Type your question '*How is this project structured?*' in the chat.



○



- ☐

- 

Start New Task



@



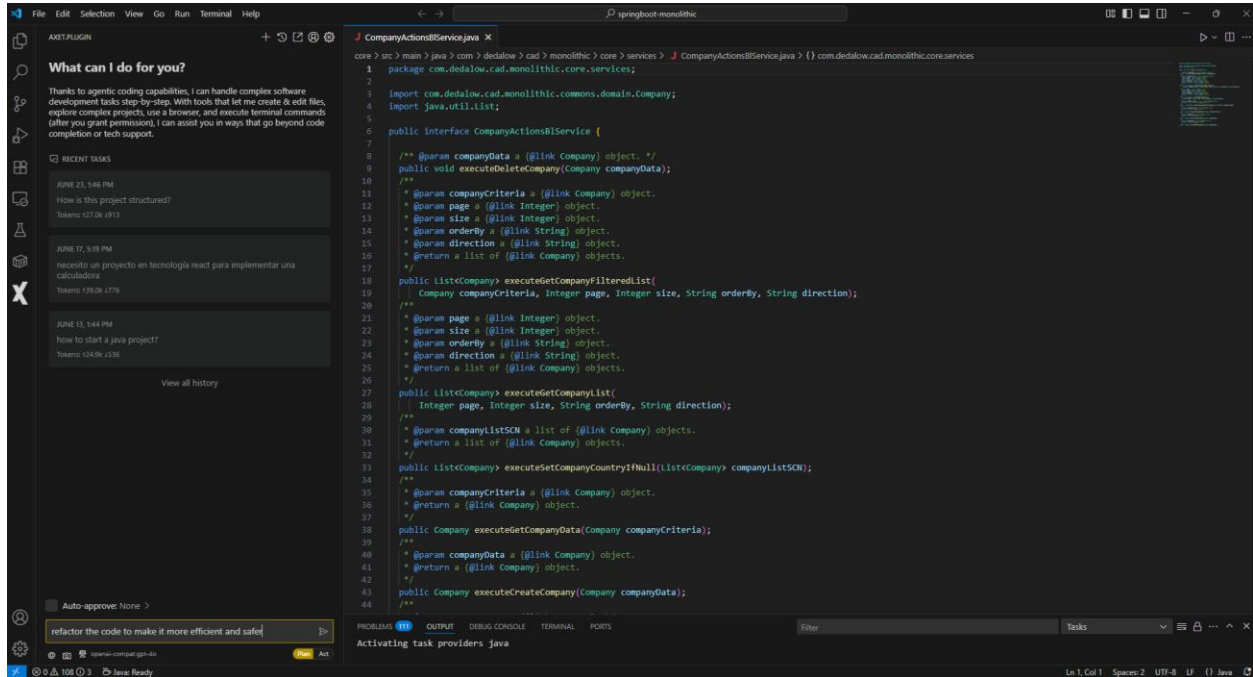
璽

openai-compat:gpt-4o

aXet.plugin goes through all the project files to analyze them and provide an answer.

You can also ask aXet about the file you have open.

Click on ‘New Task’, ask .plugin to perform a new task; e.g. refactor the code to make it more efficient and safer and change to act mode.



If .plugin needs access to any folder or file, it will request it.

Task



refactor the code to make it more efficient and safer

Tokens: ↑ 13.3k ↓ 201

Context Window: 13.5k 128.0k

34.3 kB

Git must be installed to use checkpoints. [See here for instructions.](#)

Checkpoint Compare Restore

✓ API Request



The user has requested a refactor of the code to make it more efficient and safer. The open file in the editor is `CompanyActionsBIService.java`, located in the `core/src/main/java/com/dedalow/cad/monolithic/core/services/` directory. This file is likely the primary focus for the refactor.

To proceed, I need to:

1. Analyze the contents of `CompanyActionsBIService.java` to understand its current implementation, identify inefficiencies, and pinpoint areas that could be made safer.
2. Check for dependencies or related files that might be impacted by changes to this service.
3. Formulate a plan for refactoring based on the analysis.

I will start by reading the contents of `CompanyActionsBIService.java` to gather the necessary context.

Axet wants to read this file:

...dalow/cad/monolithic/core/services/CompanyActionsBIService.java

☐ Auto-approve: None >

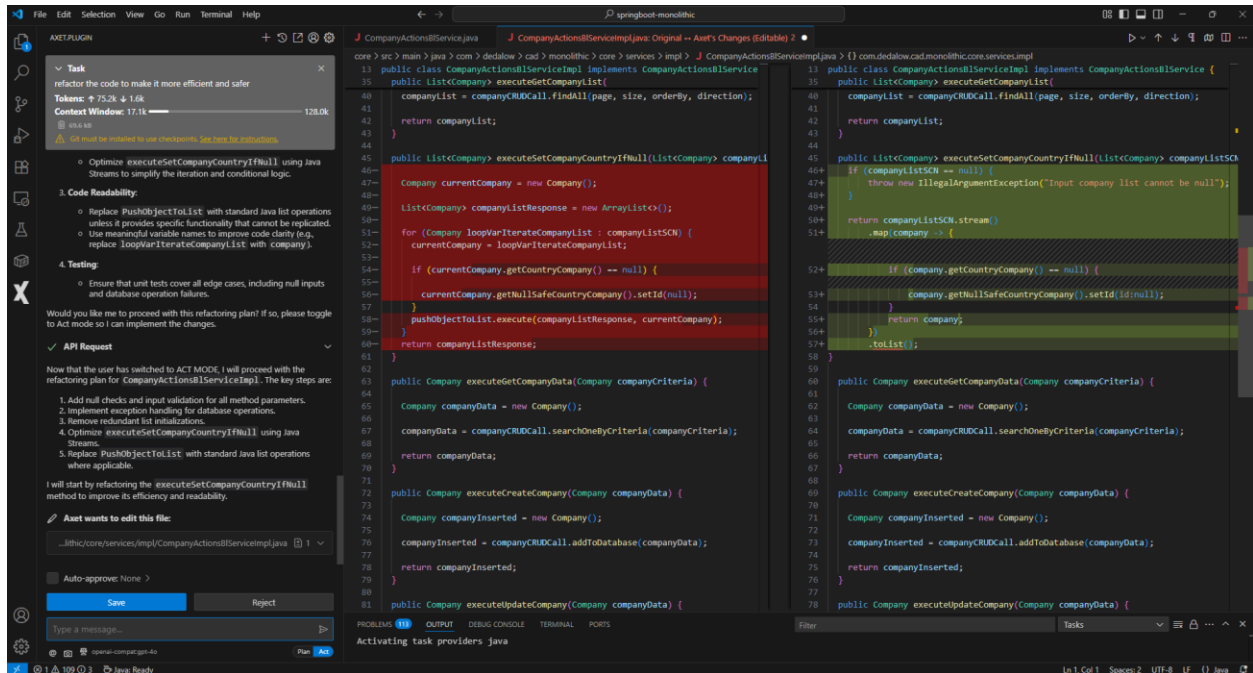
Approve

Reject

Type a message...



Once this task is finished, you can see the original file and in parallel the suggested changes.



The next step is to review the changes provided by aXet. At this point, you can click 'Save' to accept the changes and the file will be automatically modified or you can click 'Reject' to reject them and continue working.

If still have any doubts or other tasks must be executed, can keep on asking questions on the same topic and aXet will provide the help needed.

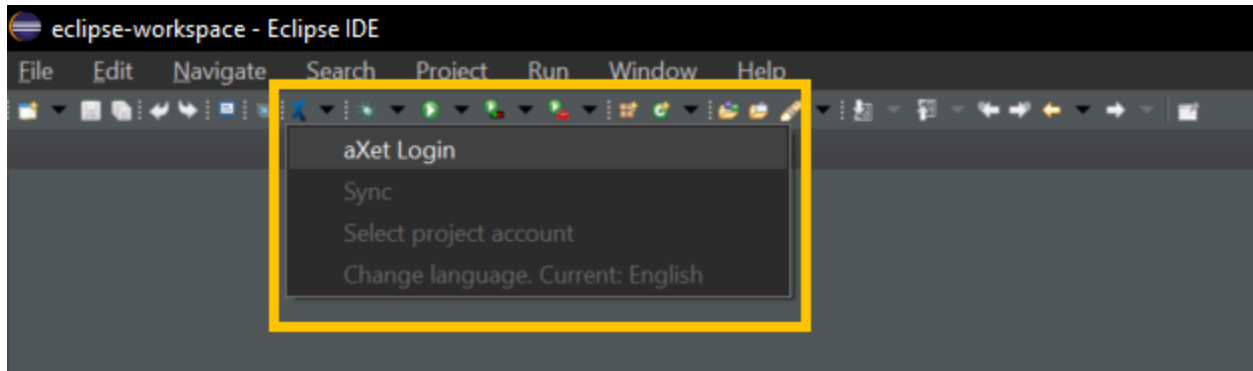
Use aXet.plugin in Eclipse

Before starting to use the **plugin** in Eclipse, you must log into **aXet**.

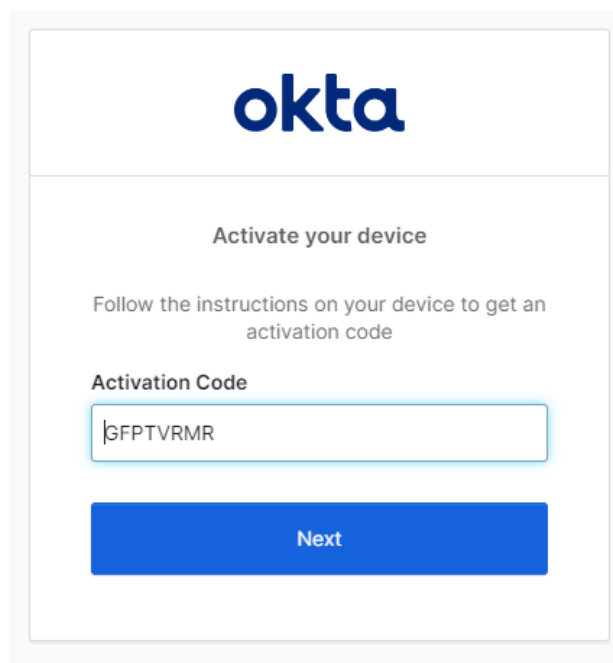
Remember that you must have an active user to use it.

These are the steps you must follow:

1. Click on 'aXet' menu in the toolbar and select 'aXet Login' item.



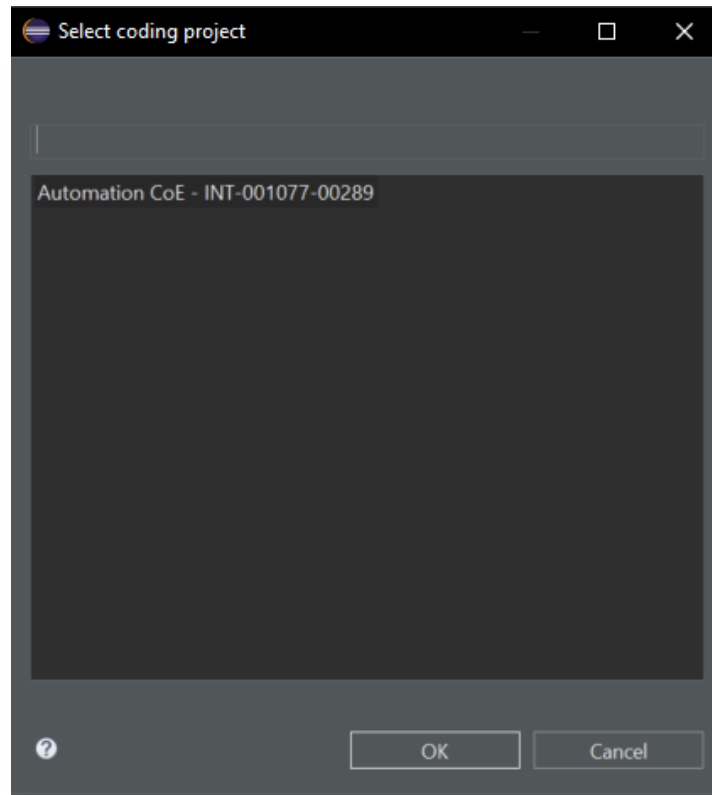
2. A window from Okta opens in the browser with an activation code. Click on the 'Next' button.



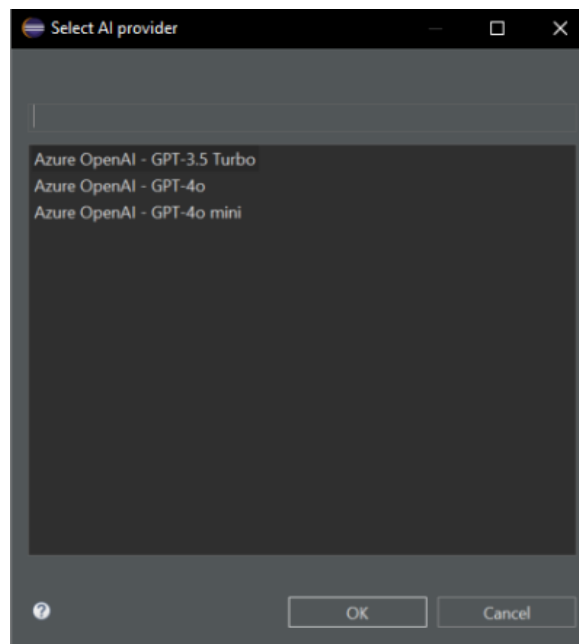
If you are logged into Okta, return to Eclipse to continue with the login process.

If you are not logged into Okta, you must follow the steps to do it before continuing with the login in Eclipse.

3. Return to Eclipse to select the project you are going to work on.

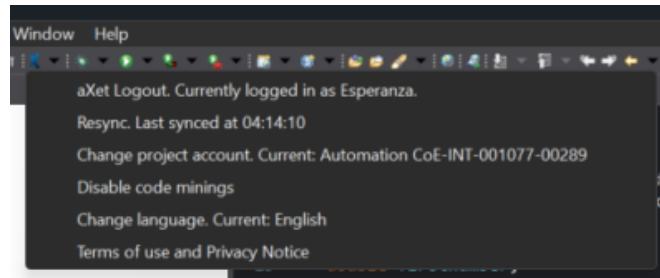


4. Select the AI model you want to use. You can change it later.



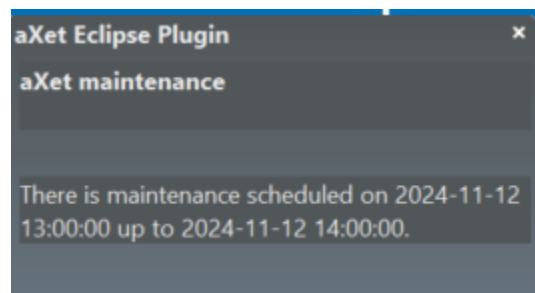
- 5.
6. The first time you access .plugin you must accept the terms of use. On the aXet.gaia page, you can read and accept these terms of use. Until you accept them, you will not be able to start using it.
7. Return to the IDE to start working with aXet in Eclipse.

Before starting to use the .plugin, let's look at the available options in 'aXet' menu.

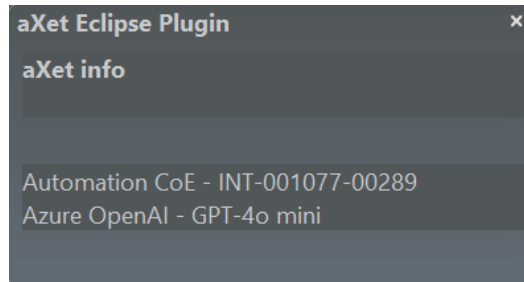


- **aXet Logout.** It allows you to logout in **aXet** platform.
- **Resync.** Force the synchronization of the .plugin with the **aXet** platform, this includes projects, users, groups ...
- **Change project account.** It allows you to switch projects. If you are registered in multiple projects, you can select which one you are going to work on.
- **Disable code minings.** It allows you to enable and disable the use of code minings (CodeLense).
- **Change language.** By default, **aXet** messages will be displayed in the same language that is set in Eclipse, but clicking this button allows you to change it.
- **Terms and Conditions.** It allows you to revisit to consult the terms of use and privacy that you have accepted to start using the extension.

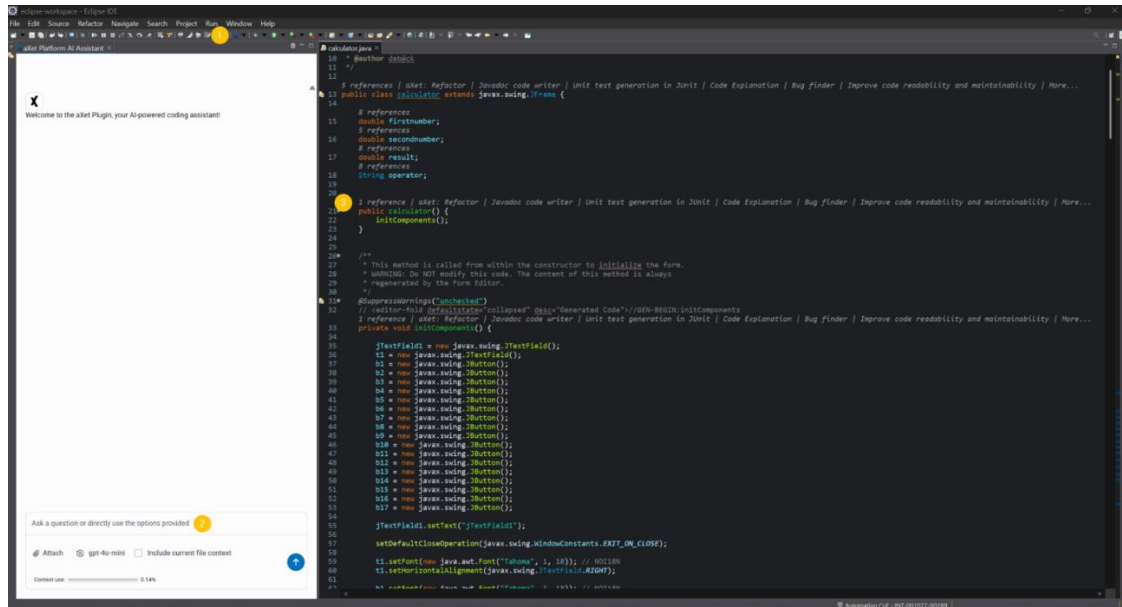
You will also see a notification if **aXet.gaia** is under maintenance or if there are scheduled maintenance actions. The scheduled maintenance notifications from **aXet** include information on the date, time, and time zone.



Click on the icon with the name of the project you are working on, and a window will show you information about the project and the selected AI model.



Let's review the Eclipse interface when you're ready to work with **.plugin**.



These are the points highlighted in the image:

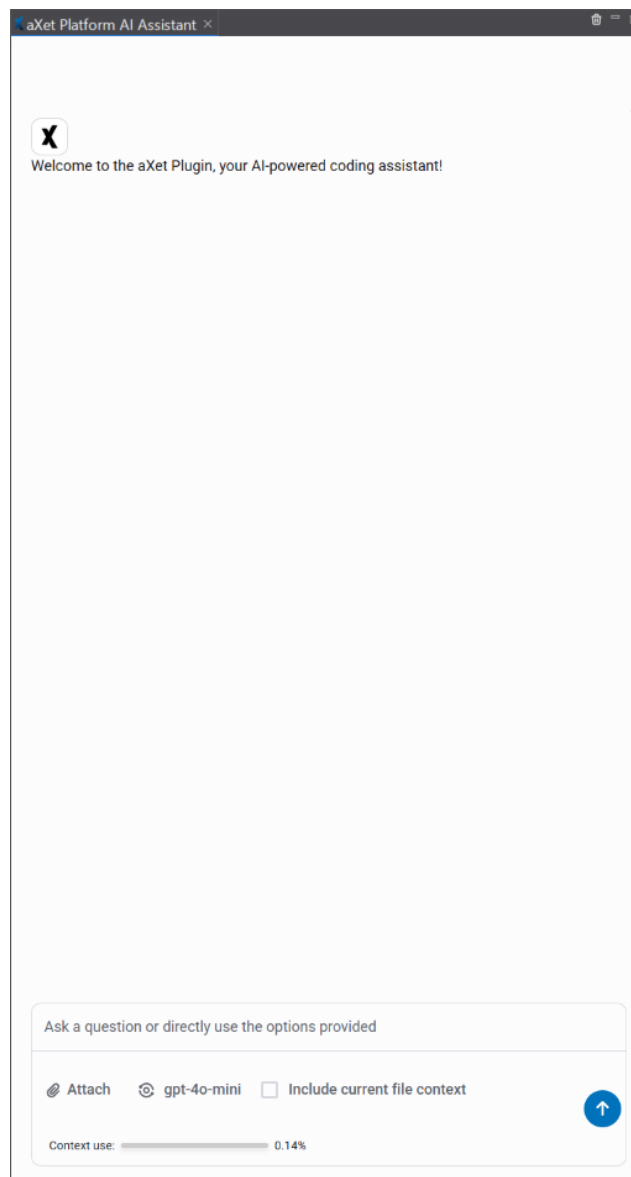
1. **aXet.plugin shortcut.** When the plugin has been successfully installed in Eclipse, you can find the shortcut in the toolbar.
2. **aXet.plugin chat.** Ask an open question and get the answer from **aXet**. You can use collections of documents to provide the context to create a more relevant, accurate, and tailored responses. You can also see information about the tokens used in queries to manage resource usage efficiently.
3. **aXet.plugin CodeLens.** When the **.plugin** has been installed correctly and you are logged into **aXet** platform, you can use CodeLens with predefined and appropriate actions on each of the methods, classes, and functions in Java files. You can use collections of documents to provide the context to create a more relevant, accurate, and tailored responses.

You have two ways to interact with **.plugin** to ask questions to **aXet**. You can ask free questions using the chat or you can use CodeLens with predefined and appropriate

actions on each of the methods, classes, and functions only in Java files. Let's see how to use each of these two ways.

Use chat in Eclipse

When the **.plugin** has been installed correctly and you are logged into **aXet** platform, you can start asking questions to **aXet** using the free chat even if you are not working on a project.



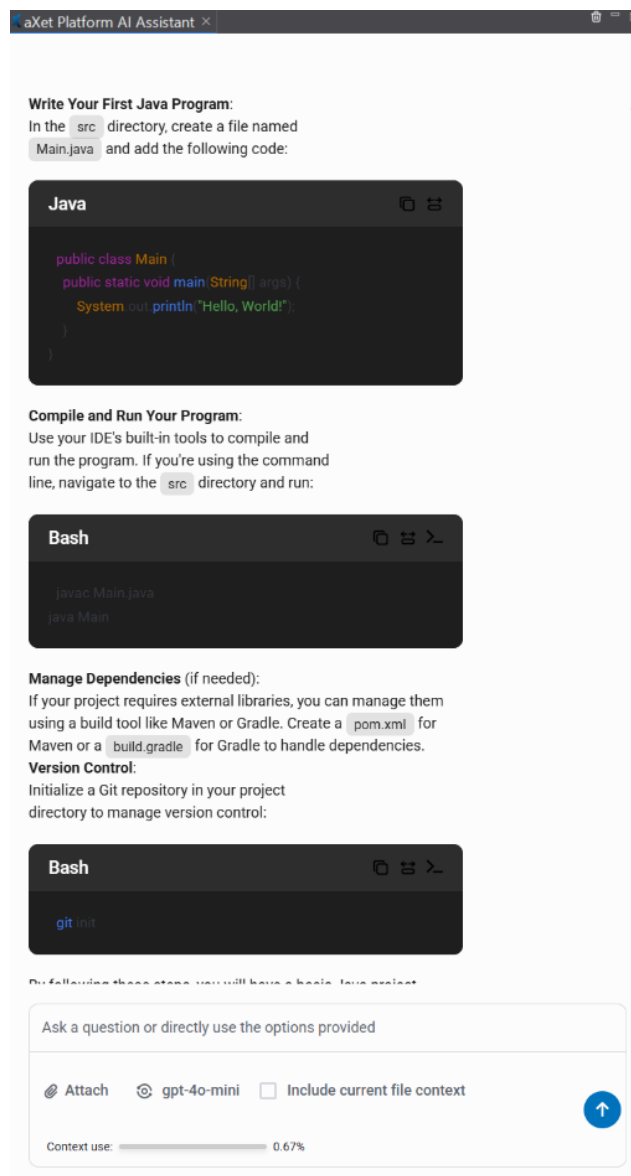
Ask an open question and get the answer from **aXet**. If the **aXet.gaia** project you have connected to has collections set up, whether public for all project users or private because

you created them, you will be able to use them here to provide context to your questions. To do this, you must click the 'Attach' button and check the ones you want to use.

Select the AI model from the available ones.

You can add the file that is open in the IDE to provide context to the conversation you are going to start in the chat. The checkbox 'Include current file context' allows you to select/deselect.

You can also see information about the tokens used in queries to manage resource usage efficiently.



You can find 2 buttons in the explanation.

Example Code

Java

```
package com.example.myproject;

public class Main {
    public static void main(String[]
args) {
        System.out.println("Hello,
World!");
    }
}
```

With the first button, you can copy and paste the displayed information.

With the second button, you can directly add the information to the file you are working with.

If you still have any doubts, you can continue asking questions on the same topic and **aXet** will provide you with the help you need.



Click on the trash can icon if you want to clear the chat content and you will be able to start a new conversation with **aXet**.

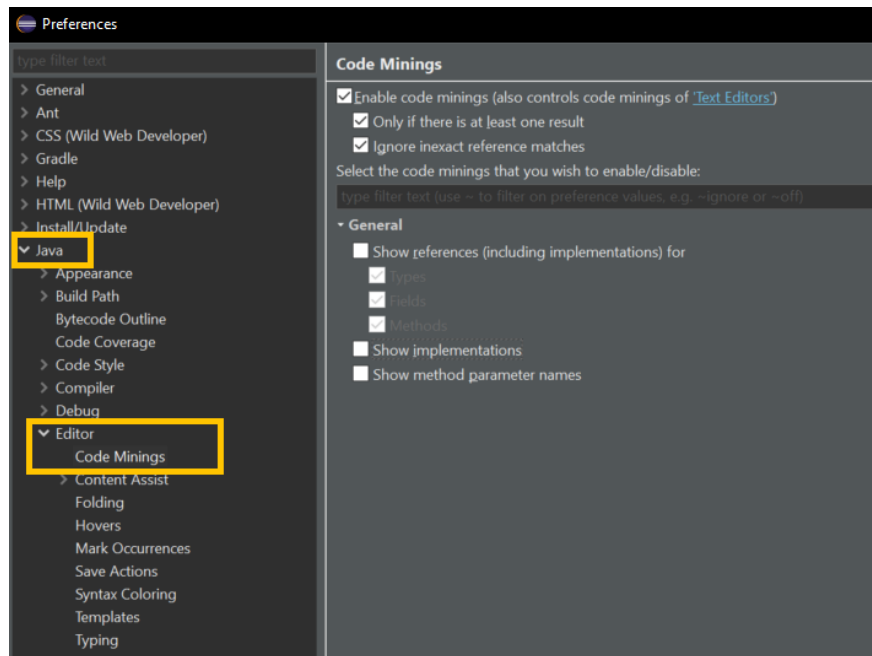
The next time you open VS Code and access the aXet extension, you will see the history of all the questions you have asked.

Use CodeLens in Eclipse

When the **.plugin** has been installed correctly and you are logged into **aXet** platform, you can use CodeLens with predefined and appropriate actions on each of the methods, classes, and functions in Java files.

Before opening your Java project, make sure that CodeLens is available for use in Eclipse. These are the steps to follow:

1. Open the 'Window' menu and select 'Preferences'.
2. In the 'Java' section, select 'Editor' and click on 'Code Minings'.
3. Make sure that the following checks are marked.



Now, you are ready to open up a Java project, and you'll see the CodeLens options within the code.

```
CampaignManagerDBDataSourceConfig.java
1 package com.dedalow.cad.monolithic.config;
2
3 import com.zaxxer.hikari.HikariDataSource;
4
5 @Configuration
6 @EnableJpaRepositories({
7     basePackages = {"com.dedalow.cad.monolithic.commons.repository.campaignManagerDB"},
8     entityManagerFactoryRef = "campaignManagerDBEntityManagerFactory",
9     transactionManagerRef = "campaignManagerDBTransactionManager")
10
11 // aXet: Refactor | Code Explanation | Bug finder | Unit test generation in JUnit | Javadoc code writer | Improve code readability and maintainability | More...
12 public class CampaignManagerDBDataSourceConfig {
13
14     @Value("${api.campaignManagerDB.datasource.username}")
15     private String username;
16
17     @Value("${api.campaignManagerDB.datasource.password}")
18     private String password;
19
20     @Value("${api.campaignManagerDB.datasource.url}")
21     private String dataSourceUrl;
22
23     @Value("${api.campaignManagerDB.datasource.schema}")
24     private String schema;
25
26     @Value("${api.campaignManagerDB.datasource.className}")
27     private String dataSourceClassName;
28
29     @Value("${api.campaignManagerDB.datasource.connectionTimeout}")
30     private int connectionTimeout;
31
32     @Value("${api.campaignManagerDB.datasource.maxLifetime}")
33     private int maxLifetime;
34
35     @Value("${api.campaignManagerDB.datasource.maxPoolSize}")
36     private int maxPoolSize;
37
38     @Value("${api.campaignManagerDB.datasource.ddlStrategy}")
39     private String ddlStrategy;
40
41     @Value("${api.campaignManagerDB.datasource.entityCopyObserver}")
42     private String entityCopyObserver;
43
44     @Value("${api.campaignManagerDB.datasource.dialect}")
45     private String hibernateDialect;
46
47 }
```

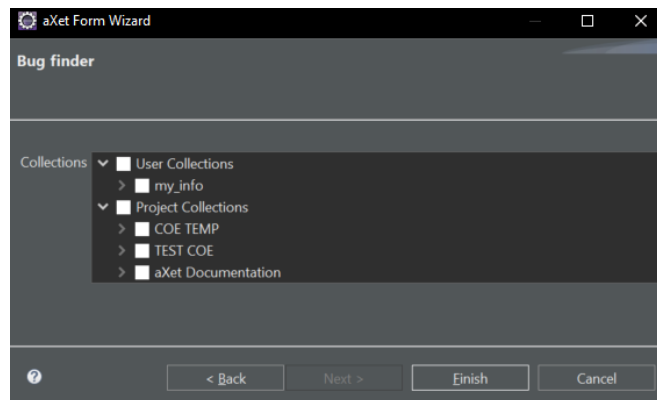
If not all actions can be displayed, the ‘More...’ option is available. Click on it and you will see the rest of the available actions.

Let’s ask **.plugin** to inspect the code using the ‘Bug finder’ option.

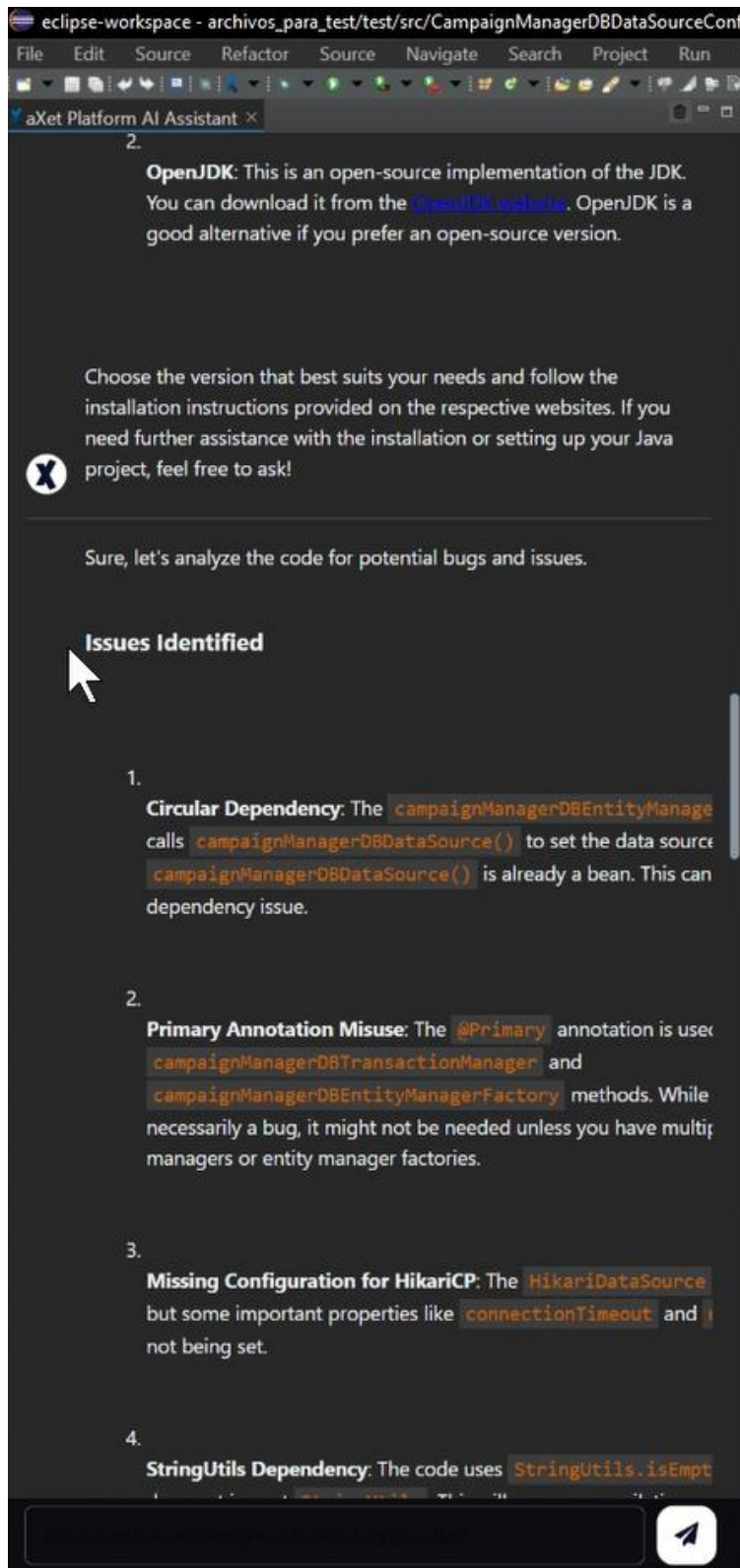
Depending on the task, you will need to provide more information for **aXet** to execute it.

You can use collections of documents to provide context to your request selecting from the list. Check the ones you want to use, could be public for all the users of the project or your private ones.

This collections of documents must be create and manage in **aXet.gaia**.



You will get the answer from **aXet**.



.plugin provides a natural language explanation of the issues in this code and how to fix them.

Regarding the explanation:

- You can copy and paste the displayed information.
- You can directly add the information to the file you are working with.

The next time you open VS Code and access the aXet extension, you will see the history of all the questions you have asked.

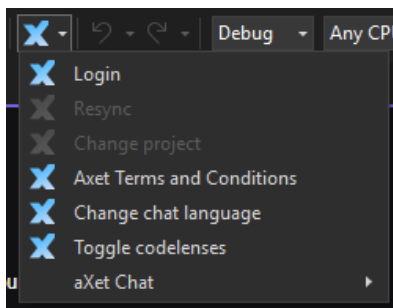
Use aXet.plugin in Microsoft VS

Before starting to use the **plugin** in Microsoft Visual Studio Professional, you must log into **aXet**.

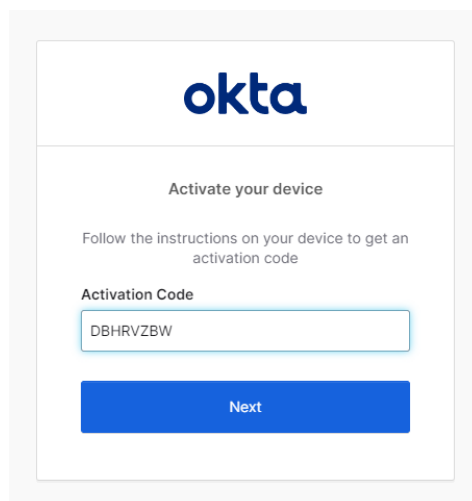
Remember that you must have an active user to use it.

These are the steps you must follow:

1. Click on aXet menu and select 'Login' item.



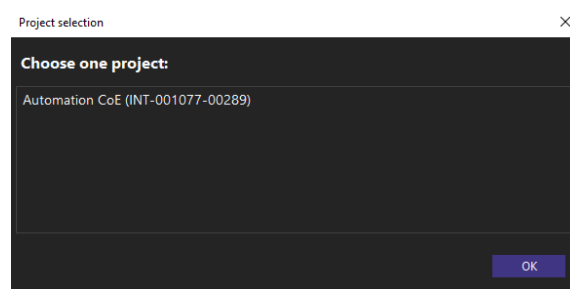
2. A window from Okta opens in the browser with an activation code. Click on the 'Next' button.



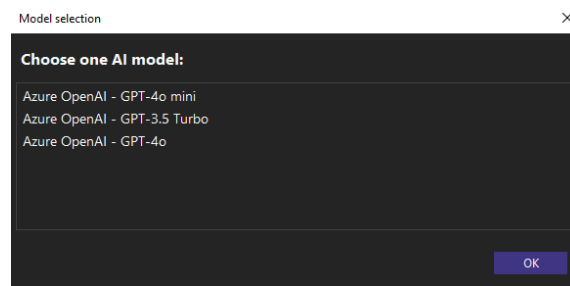
If you are logged into Okta, return to the Microsoft Visual Studio Professional interface to continue with the login process.

If you are not logged into Okta, you must follow the steps to do it before continuing with the login in Microsoft Visual Studio Professional.

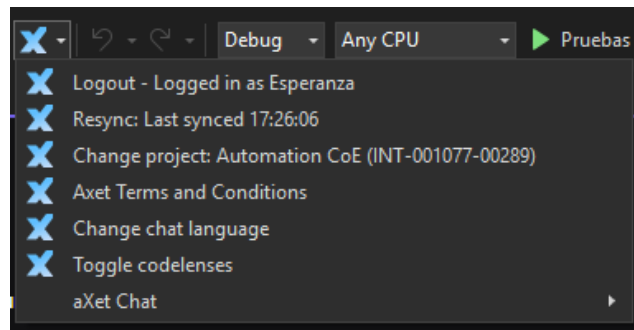
3. Return to Microsoft Visual Studio Professional to select the project you are going to work on.



4. Select one AI model. You can change it later on in case you need it.



5. You are logged in successfully.



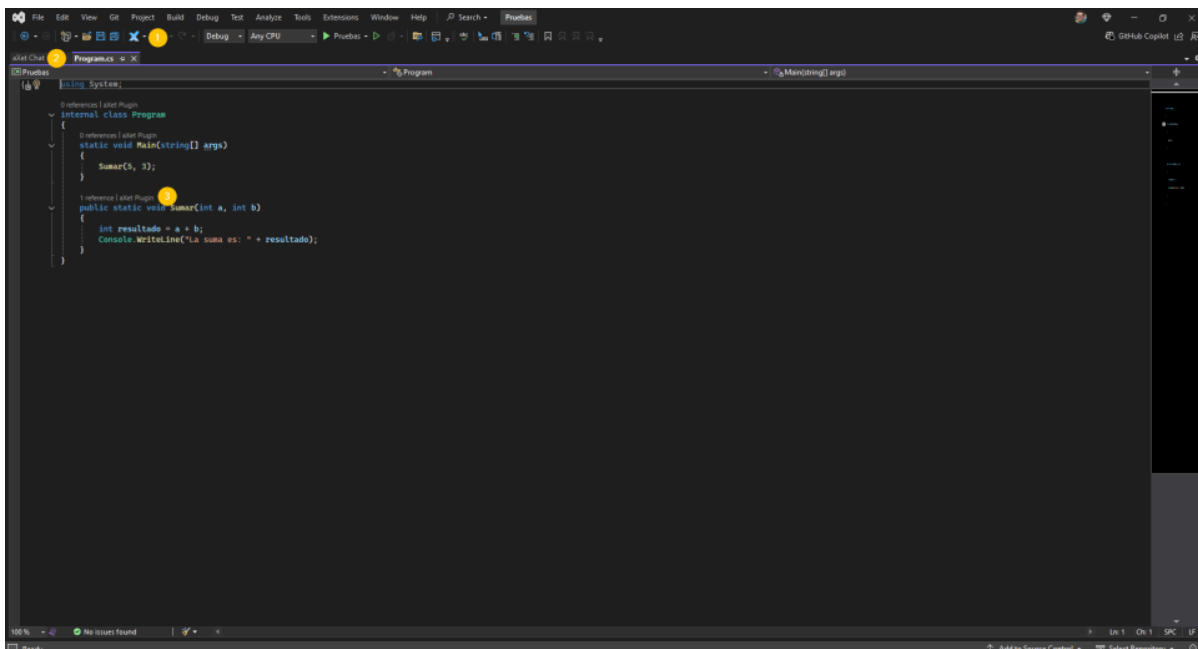
Before starting to use the extension, let's look at the available options by clicking on the 'aXet' menu.

- **Logout.** It allows you to logout in **aXet** platform.
- **Resync.** Force the synchronization of the extension with the **aXet** platform, this includes projects, users, groups ...

- **Change project.** It allows you to switch projects. If you are registered in multiple projects, you can select which one you are going to work on.
- **Axet Terms and Conditions.** It allows you to revisit to consult the terms of use and privacy that you have accepted to start using the extension.
- **Change Language:** By default, **aXet.plugin** chat messages will be displayed in the same language that is set in Microsoft Visual Studio Professional. It allows you to select the language in which **aXet.plugin** chat will respond and CodeLens will be displayed.
- **Toggle code lenses.** It allows manage CodeLens.
- **aXet chat.** It allows you:
 - **Open Chat.** It opens the chat panel to ask an open question and get the answer from **aXet**.
 - **Change Theme.** It allows you to change the theme of the chat panel.
 - **Clear Chat.** It deletes the prompts and responses in the chat panel.

You will also see a notification if **aXet.gaia** is under maintenance or if there are scheduled maintenance actions. The scheduled maintenance notifications from **aXet** include information on the date, time, and time zone.

Let's review the Microsoft Visual Studio Professional interface when you're ready to work with **.plugin**.



These are the points highlighted in the image:

1. **aXet menu.** After installing the extension, from this menu you can configure and operate with **aXet.plugin**.
2. **aXet.plugin chat.** This is the chat to ask open questions to **aXet.plugin**. You can use collections of documents to provide the context to create a more relevant, accurate, and tailored responses.
3. **aXet.plugin CodeLens.** When the **.plugin** has been installed correctly and you are logged into **aXet** platform, you can use CodeLens with predefined and appropriate actions on each of the methods, classes, and functions in .NET projects files. You can use collections of documents to provide the context to create a more relevant, accurate, and tailored responses.

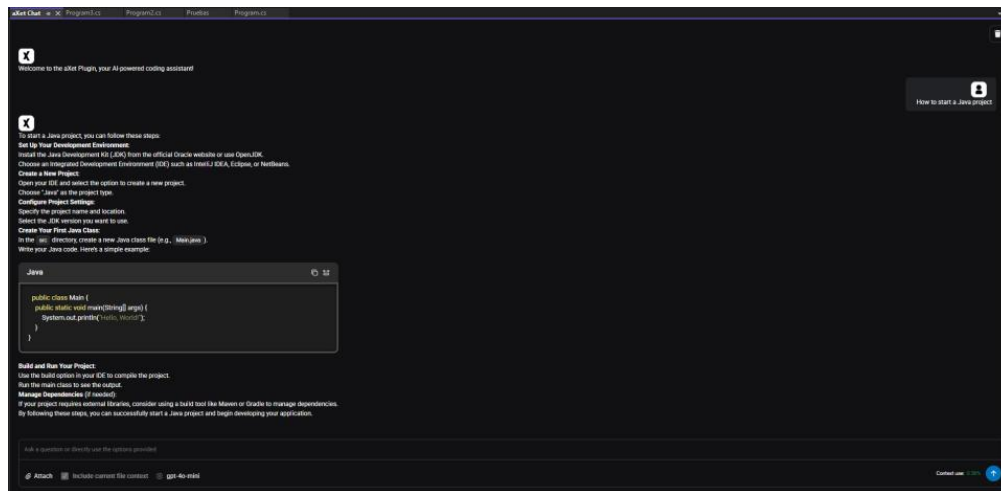
You have two ways to interact with **.plugin** to ask questions to **aXet**. You can ask free questions using the chat or you can use CodeLens with predefined and appropriate actions on each of the methods, classes, and functions in your coding files. Let's see how to use each of these two ways.

Document collections are created and managed in aXet.gaia.

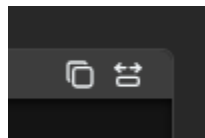
Use chat in Microsoft VS

When the **.plugin** has been installed correctly and you are logged into **aXet** platform, you can start asking questions to **aXet** using the free chat even if you are not working on a project.

Ask an open question and get the answer from **aXet**. If the **aXet.gaia** project you have connected to has collections set up, whether public for all project users or private because you created them, you will be able to use them here to provide context to your questions. To do this, you must click the '*Attach*' button and check the ones you want to use.



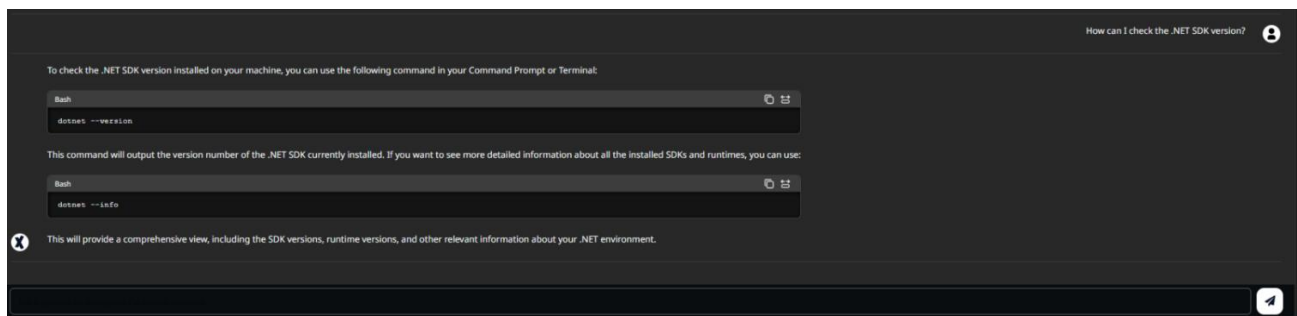
You can find 2 buttons in the explanation.



With the first button, you can copy and paste the displayed information.

With the second button, you can directly add the information to the file you are working with. If there is a code selection in the editor, you can overwrite that selection with the chat content. If there is no selection, it will be concatenated at the position where the cursor is located.

If you still have any doubts, you can continue asking questions on the same topic and **aXet** will provide you with the help you need.

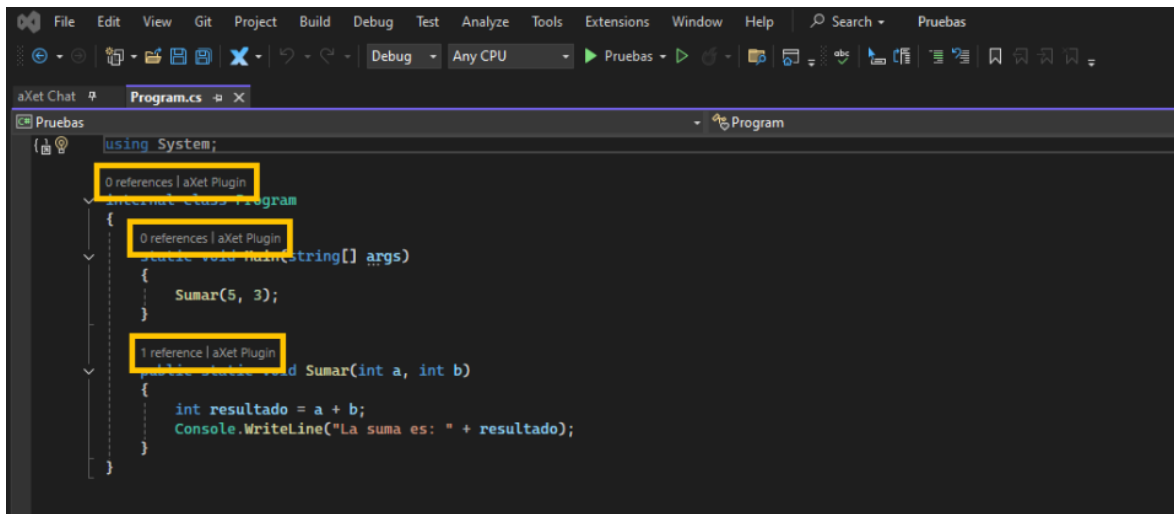


The next time you open Microsoft Visual Studio Professional and access the aXet extension, you will see the history of all the questions you have asked.

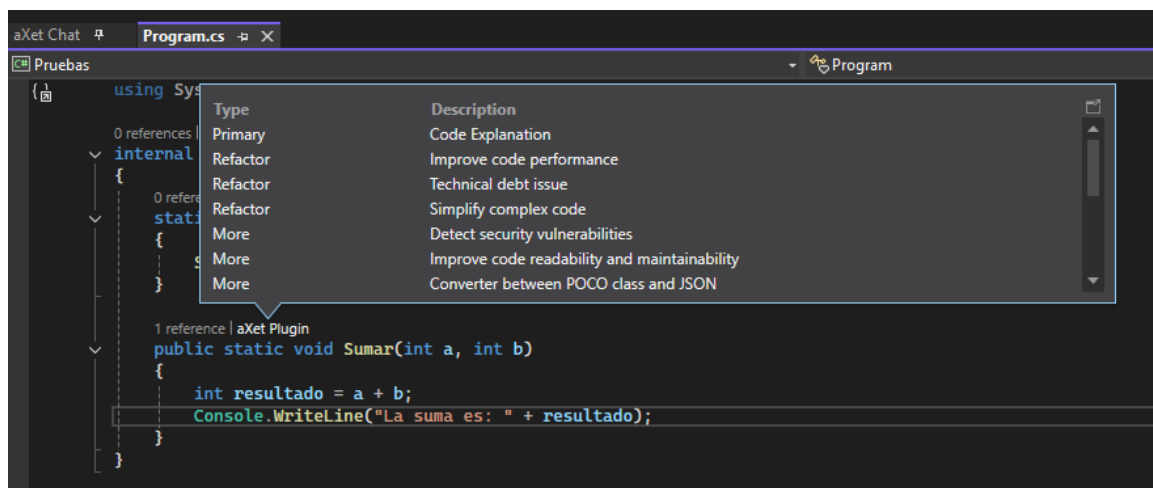
Use CodeLens in Microsoft VS

When the **.plugin** has been installed correctly and you are logged into **aXet** platform, you can use CodeLens with predefined and appropriate actions on each of the methods, classes, and functions in .NET projects files.

Open your development project and check that you have suggestions from **aXet**.



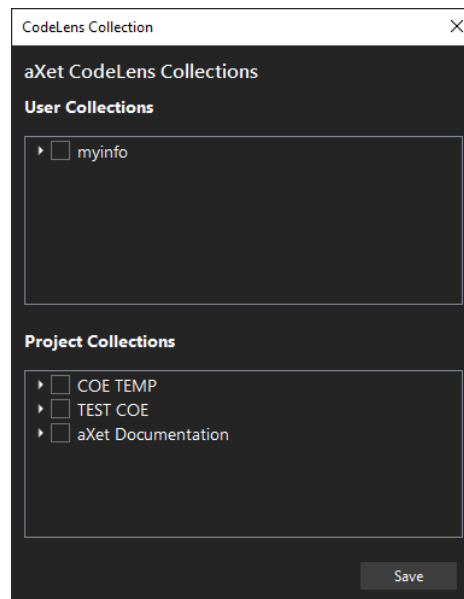
Click on '*aXet Plugin*' and select the option you are interested in as you can see in the image below.



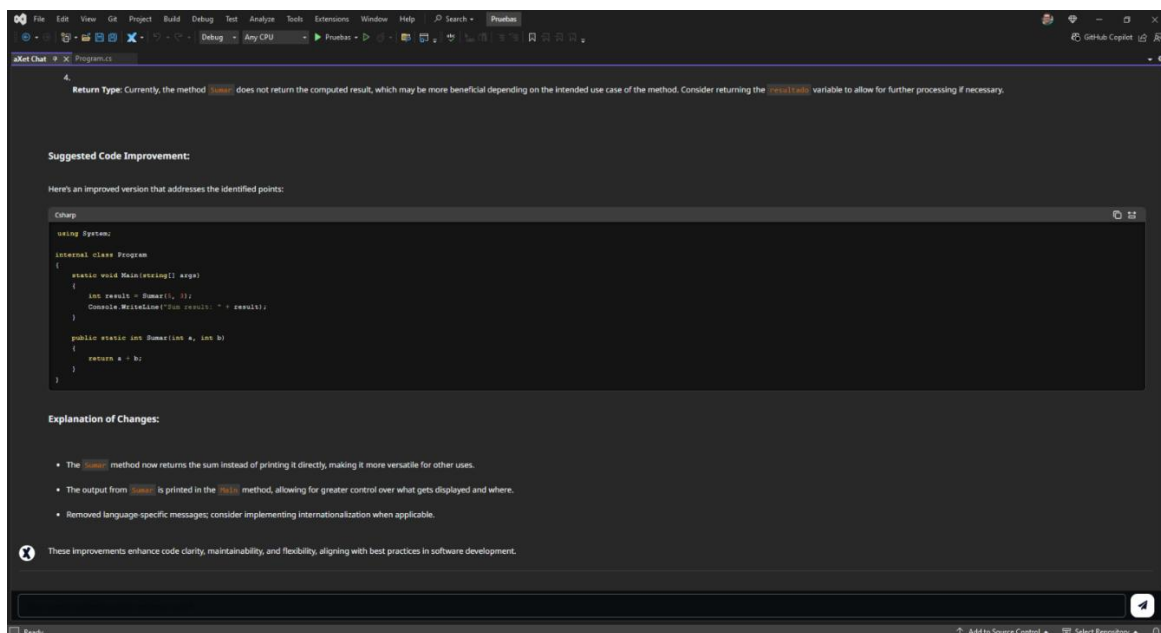
Let's ask **.plugin** to inspect the code using the '*Bug finder*' option. Depending on the task, you will need to provide more information for **aXet** to execute it.

You can use collections of documents to provide context to your request selecting from the list. Check the ones you want to use, could be public for all the users of the project or your private ones.

This collections of documents must be create and manage in **aXet.gaia**.



You will get the answer from **aXet**.



.plugin provides a natural language explanation of the issues in this code and how to fix them.

Regarding the explanation:

- You can copy and paste the displayed information.
- You can directly add the information to the file you are working with.

The next time you open Microsoft Visual Studio Professional and access the aXet extension, you will see the history of all the questions you have asked.

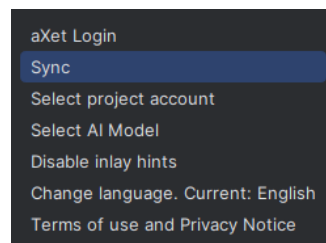
Use aXet.plugin in IntelliJ

Before starting to use the **plugin** in IntelliJ, you must log into **aXet**.

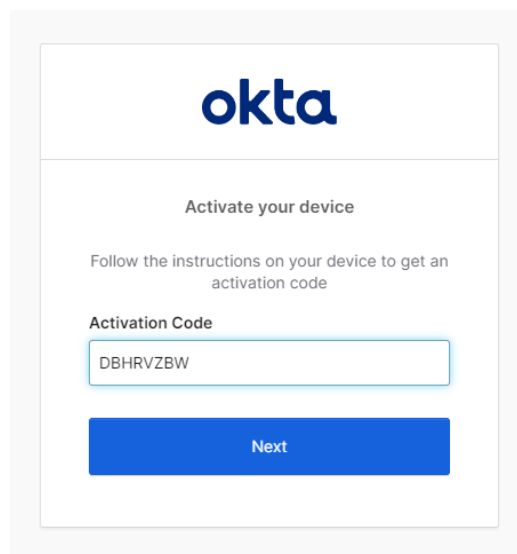
Remember that you must have an active user to use it.

These are the steps you must follow:

1. Click on **aXet** menu and select 'Login' item.



2. A window from Okta opens in the browser with an activation code. Click on the 'Next' button.



If you are logged into Okta, return to the IntelliJ interface to continue with the login process.

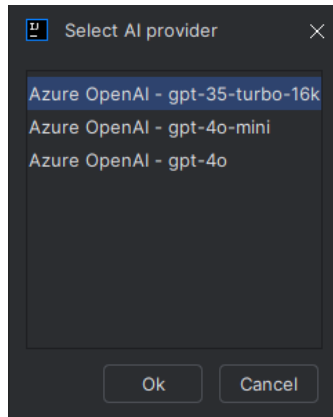
If you are not logged into Okta, you must follow the steps to do it before continuing with the login in IntelliJ.

3. Return to IntelliJ to select the **aXet** project you are going to work on.

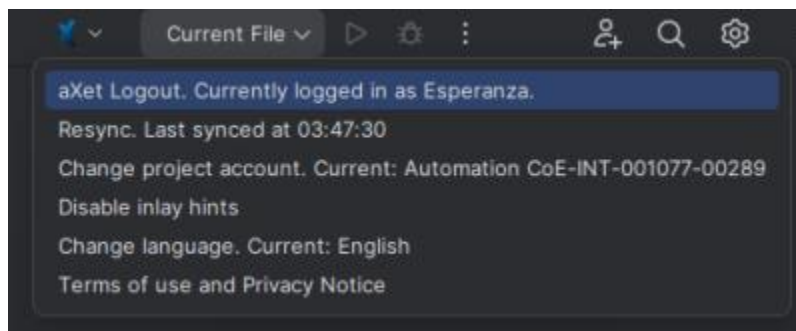
If you are registered in more than one project on **aXet.gaia**, you will be able to select which one you are going to work on.

If you are registered in only one, it is not necessary to select it.

4. Select one AI model. You can change it later on in case you need it.



5. You are logged in successfully.



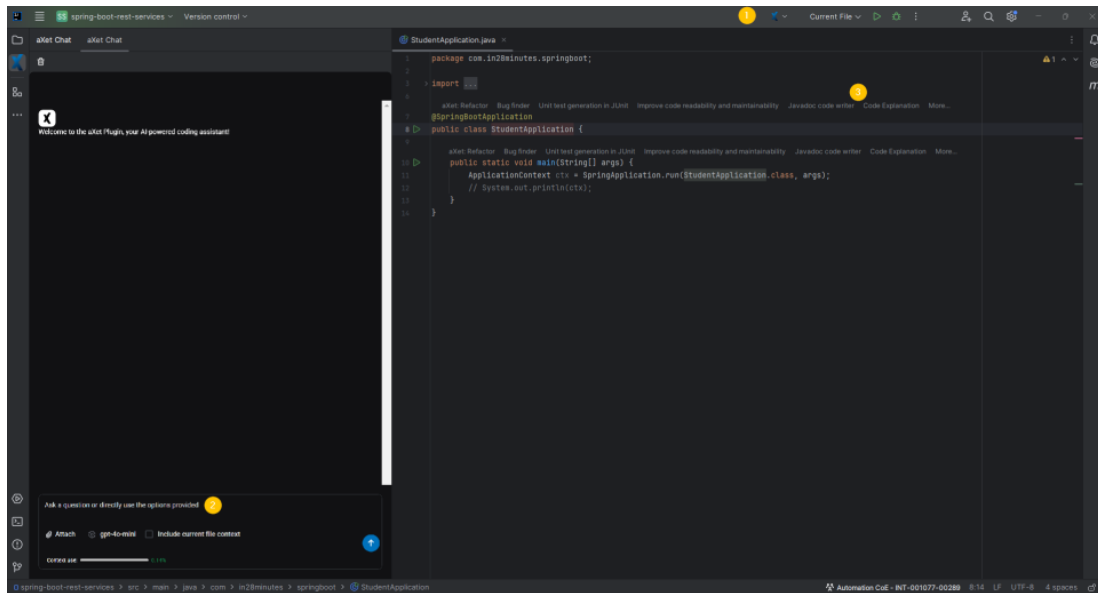
Before starting to use the extension, let's look at the available options by clicking on the 'aXet' menu.

- **Logout.** It allows you to logout in **aXet** platform.
- **Resync.** Force the synchronization of the extension with the **aXet** platform, this includes projects, users, groups ...
- **Change project account.** It allows you to switch projects. If you are registered in multiple projects, you can select which one you are going to work on.
- **Disable inlay hints.** It allows enable/disable inlay hints.
- **Change Language:** By default, **aXet.plugin** chat messages will be displayed in english. It allows you to select the language in which **aXet.plugin** chat will respond and inlay hints will be displayed.

- **Terms of use and Private Notice.** It allows you to revisit to consult the terms of use and privacy that you have accepted to start using the extension.

You will also see a notification if **aXet.gaia** is under maintenance or if there are scheduled maintenance actions. The scheduled maintenance notifications from **aXet** include information on the date, time, and time zone.

Let's review the IntelliJ interface when you're ready to work with **.plugin**.



These are the points highlighted in the image:

1. **aXet menu.** After installing the extension, from this menu you can configure and operate with **aXet.plugin**.
2. **aXet.plugin chat.** This is the chat to ask open questions to **aXet.plugin**. You can use collections of documents to provide the context to create a more relevant, accurate, and tailored responses. You can also see information about the tokens used in queries to manage resource usage efficiently.
3. **aXet.plugin inlay hints** When the **.plugin** has been installed correctly and you are logged into **aXet** platform, you can use inlay hints with predefined and appropriate actions on each of the methods, classes, and functions. You can use collections of documents to provide the context to create a more relevant, accurate, and tailored responses.

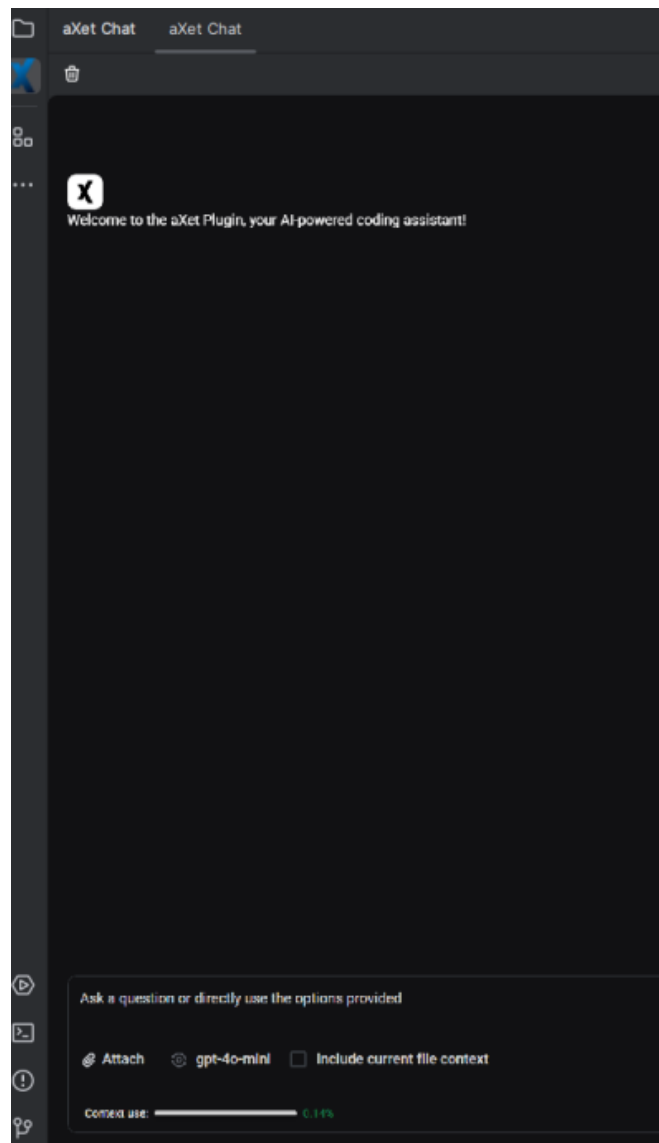
You have two ways to interact with **.plugin** to ask questions to **aXet**. You can ask free questions using the chat or you can use inlay hints with predefined and appropriate

actions on each of the methods, classes, and functions in your coding files. Let's see how to use each of these two ways.

Document collections are created and managed in aXet.gaia.

Use chat in IntelliJ

When the **.plugin** has been installed correctly and you are logged into **aXet** platform, you can start asking questions to **aXet** using the free chat.



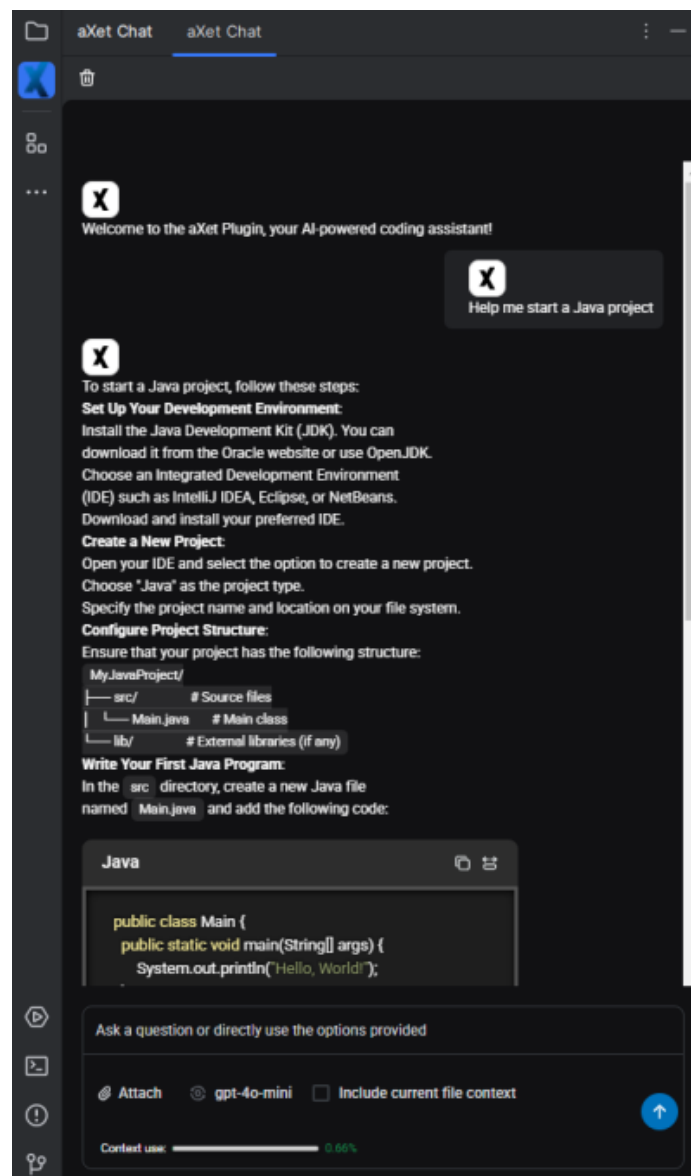
Ask an open question and get the answer from **aXet**. If the **aXet.gaia** project you have connected to has collections set up, whether public for all project users or private because

you created them, you will be able to use them here to provide context to your questions. To do this, you must click the *'Attach'* button and check the ones you want to use.

Select the AI model from the available ones.

You can add the file that is open in the IDE to provide context to the conversation you are going to start in the chat. The checkbox *'Include current file context'* allows you to select/deselect.

You can also see information about the tokens used in queries to manage resource usage efficiently.



You can find 2 buttons in the explanation.

Here is a minimal example to help you get started with the `Calculator` class:

```
Java  
```

```
package calculation;

import javax.swing.*;
import java.awt.event.*;

public class Calculator extends JFrame {
    // Add your components and logic here
    as you have in your provided FILE

    public Calculator() {
        // Initialize your components and
        layout here
        setTitle("Calculator");
        setSize(300, 400);

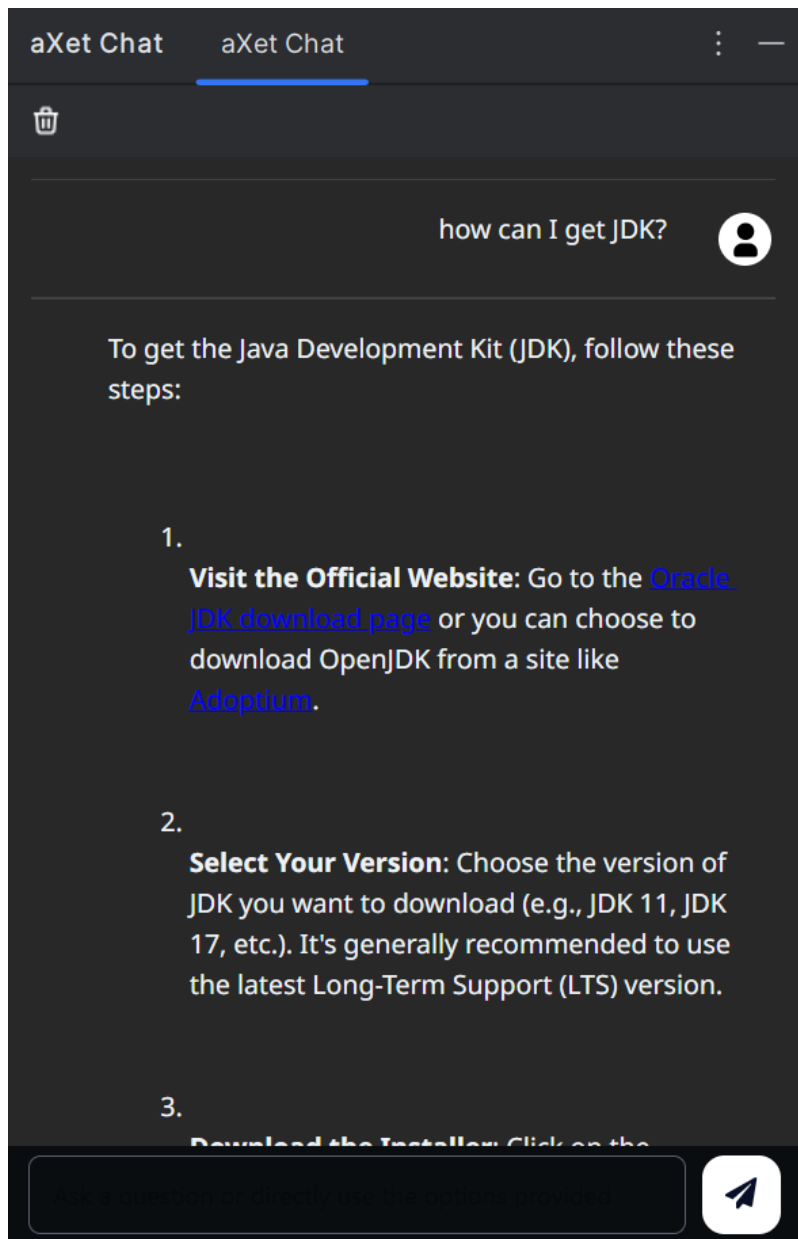
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setVisible(true);
    }

    public static void main(String[] args)
    {
```

With the first button, you can copy and paste the displayed information.

With the second button, you can directly add the information to the file you are working with.

If you still have any doubts, you can continue asking questions on the same topic and **aXet** will provide you with the help you need.



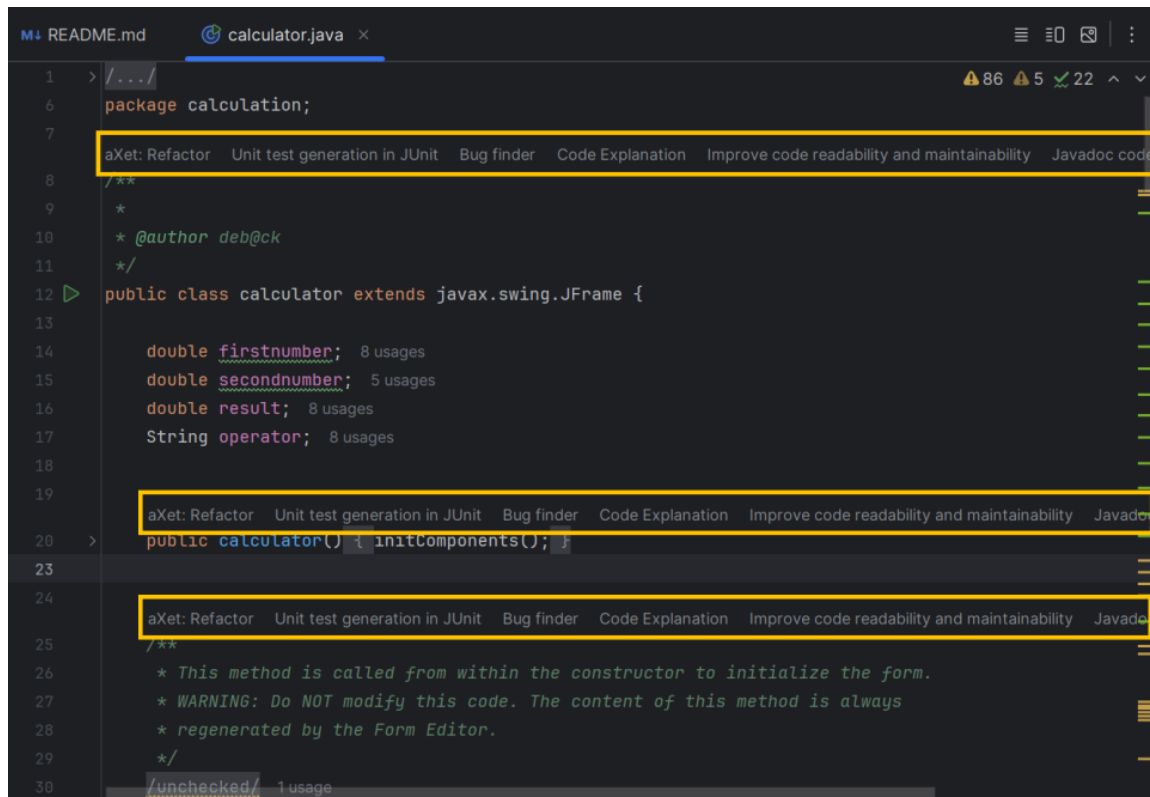
Click on the trash can icon if you want to clear the chat content and you will be able to start a new conversation with **aXet**.

The next time you open IntelliJ and access the aXet extension, you will see the history of all the questions you have asked.

Use Inlay Hints in IntelliJ

When the **.plugin** has been installed correctly and you are logged into **aXet** platform, you can use inlay hints with predefined and appropriate actions on each of the methods, classes, and functions in your coding files.

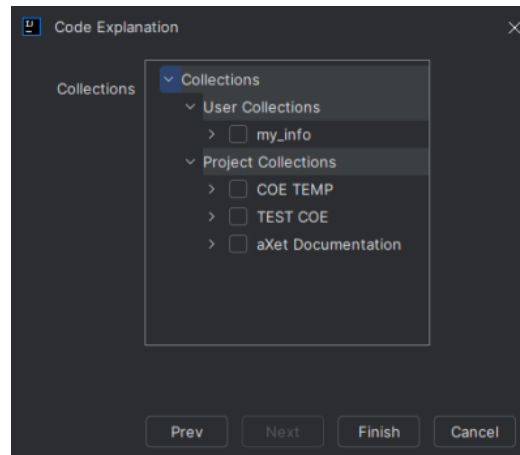
Open your development project and check that you have suggestions from **aXet**.



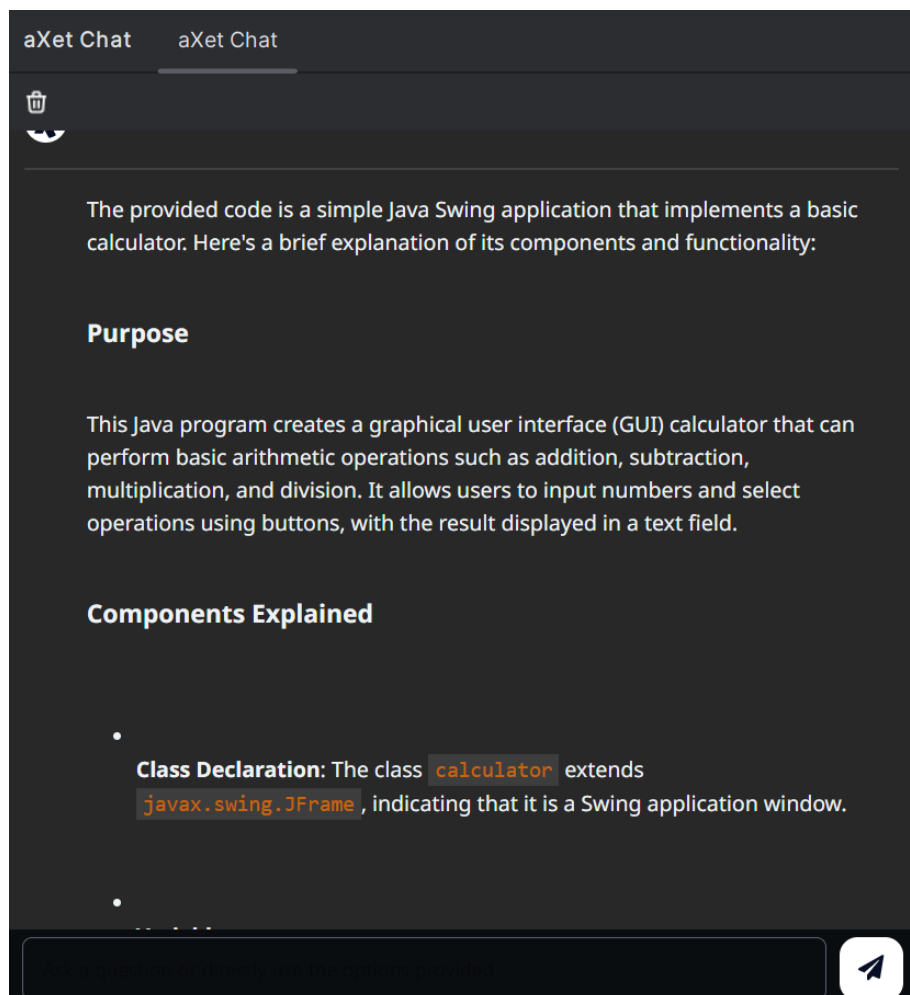
Select the task you want to perform and **aXet** will execute it. For example, let's ask to 'Code Explanation'. Depending on the task, you will need to provide more information for **aXet** to execute it.

You can use collections of documents to provide context to your request selecting from the list. Check the ones you want to use, could be public for all the users of the project or your private ones.

This collections of documents must be create and manage in **aXet.gaia**.



You will get the answer from **aXet**.



.plugin provides a natural language explanation of the code.

Regarding the explanation:

- You can copy and paste the displayed information.
- You can directly add the information to the file you are working with.

The next time you open IntelliJ and access the aXet extension, you will see the history of all the questions you have asked.

aXet.plugin - Best practices

- Keep the **.plugin** version updated to get all available functionality and improvements. This will save you time in development.

When a new version of **.plugin** is available, you will see an informational notice in the IDE so you can update it.

Follow the [link](#) to find the latest version of **aXet.plugin**.

- Choosing the right IDE and using the appropriate extension, such as **.plugin**, brings important benefits such as facilitating maintenance, scalability, documentation, testing, and the localization and resolution of errors in the code.

Nearly Decalogue of the Ideal User of AI for Developers

1. **Support Your Development:** Use **.plugin** to complement your work, such as generating code examples or suggestions, but always review and understand what it generates before integrating it into your project.
2. **Formulate Clear Problems:** Pose problems and requirements clearly and in detail. Provide context about the programming language, development environment, and specific objectives.
3. **Iterate and Refine:** If the first response is not perfect, do not hesitate to ask follow-up questions or request improvements. **.plugin** can help you iterate on solutions until they meet your needs.
4. **Verify and Test the Code:** Consider the generated code as a starting point. Always review, debug, and test any fragment thoroughly before using it in production.
5. **Promote Good Practices:** Request and follow recommendations on design patterns, SOLID principles, and other good programming practices. Use the AI to maintain clean and sustainable code.
6. **Document Clearly:** Use **.plugin** to generate or improve code documentation. Ensure that explanations are clear and understandable both for you and other developers.
7. **Optimize Performance:** Request suggestions to improve your code's performance or to identify potential bottlenecks. **.plugin** can help you consider alternative approaches or optimization techniques.

aXet.plugin – FAQs

What is considered improper use of the asset? What consequences might there be?

aXet.plugin is designed as a question-and-answer-based asset; attempts at mass generation or the use of "prompt injection" techniques will be considered misuse and may result in the temporary or permanent deactivation of the offending user.

Is integration with other LLM models being considered?

The asset's architecture allows us to consider integration with other Large Language Models (LLMs), although we currently work with OpenAI models using the authenticated API from Coding by NTT DATA.

How can I request the inclusion of new groups of questions in the tree?

For users in EMEAL, this is the [link](#) to open a ticket requesting an improvement.

For users at INC., we will soon provide a procedure.

aXet.plugin – Glossary

This is a glossary of relevant terms:

- **Generative AI**

Generative Artificial Intelligence is a field of Artificial Intelligence that encompasses systems and algorithms capable of generating new, original, and creative content. This type of AI learns from enormous datasets and generates new similar content (text, images, music, and other types of data). It falls within a specific type of Machine Learning known as Deep Learning, which is based on the use of deep neural networks. These networks are configured in various architectures, such as Generative Adversarial Networks (GANs) or diffusion models based on Transformers.

- **Machine Learning**

Machine Learning is a branch of computer science that aims to enable computers to perform tasks without being explicitly programmed to do so. These are models that can gradually improve their accuracy based on the data provided. Its most basic application is the practice of using algorithms to process data, learn from it, and then be able to make a prediction or suggestion about something. It comprises three categories of learning: supervised, unsupervised, and reinforcement learning.

- **Supervised Learning**

Supervised Learning is a type of learning in Machine Learning models that relies on labeled data and has high accuracy and easily interpretable results, which is why it is widely used in industry. Its main drawback is that it requires data to be previously labeled (which can be costly or unfeasible in many cases).

- **Unsupervised Learning**

Unsupervised Learning is a type of learning in ML models that uses unlabeled data and, unlike Supervised Learning models, can detect hidden patterns in the data without human intervention. This includes clustering algorithms, for example, to group customers into segments and detect similar behaviors.

- **Reinforcement Learning**

Reinforcement Learning is a type of learning in ML models where the machine is guided to learn from the repetition of actions and associated rewards. The agent, representing the machine, takes actions in an environment and receives rewards when correct decisions are made. Over time, the agent learns to choose the correct

actions to maximize its reward. Reinforcement Learning from Human Feedback is the technique behind large language models like ChatGPT.

- **Neural Networks**

Neural networks are a type of computational model that teaches machines to process data in a way inspired by the human brain. It consists of a set of units, called artificial neurons, connected to transmit signals. The input information passes through the neural network (where it undergoes various operations), producing output values.

- **Deep Learning**

Deep Learning is a type of Machine Learning that uses complex neural networks with three or more layers. These neural networks attempt to emulate the behavior of the human brain—though far from matching its capacity—but allow it to "learn" from large amounts of data. While a single-layer neural network can make approximate predictions, additional hidden layers add complexity to the model and help optimize its accuracy.

- **GAN**

Generative Adversarial Networks, or GANs, are a method based on training two neural networks, one called the generator and the other the discriminator, which compete with each other to generate new instances that resemble the training data distribution.

- **LM and LLM – Large Language Models**

Language models (LM) are machine learning models that predict the next word in a sentence, considering the context of previous words. They are based on a probabilistic description of language phenomena. A colossal language model or LLM, also called a large language model, is a language model consisting of a neural network with many parameters, trained on large amounts of unlabeled text through self-supervised or semi-supervised learning. These models can be used in a wide range of natural language processing tasks, such as text creation, document summarization, or language translation.

- **NLP**

Natural Language Processing (NLP) is a field within artificial intelligence and applied linguistics that studies interactions using natural language between humans and machines. It focuses on processing human communications, breaking them into parts, and identifying the most relevant elements of the message. With Natural

Language Understanding and Generation, it aims for machines to understand, interpret, and manipulate human language.

- **Open AI**

Open AI is an artificial intelligence (AI) research lab founded in 2015 that operates with the declared intention of promoting and developing friendly and democratized AI. OpenAI systems run on a supercomputing platform based on Microsoft's Azure, and some of its best-known works are generative AIs like ChatGPT (the leading conversational AI) or DALL-E (the AI for generating images from natural language).

- **Chat-GPT**

Chat GPT is a language model developed by OpenAI that employs natural language processing (NLP) techniques to generate coherent and natural responses in real-time. Its free version uses the GPT-3.5 model, and the most current version is GPT-4.

- **Fine-Tuning**

Fine-Tuning is a training technique that involves reusing predefined and pre-trained neural network architectures. It is a process where a "fine-tuning" of some layers of the network is performed to obtain the desired outputs. In other words, certain representations of the pre-trained model are slightly adjusted to make it more relevant to the problem to be solved. This avoids training the network from scratch.

- **Prompts**

A "prompt" in AI refers to an instruction or request that a user makes to an LLM, chatbot, or AI model. This instruction can come not only in natural language (text or audio) but also in other formats like images or code. The careful and concise preparation of these instructions is known as "prompting," and it ultimately aims to obtain the desired results or responses from LLMs or generative models of images and videos.

- **Token**

In the context of large language models or LLMs, a token is a basic unit of information for text processing. These tokens generally correspond to words or phrases, but they can also be punctuation marks or numbers. Transforming text into chains of tokens allows LLMs to process user prompts and generate corresponding responses. A model like GPT-3.5 could handle 4,096 tokens or around 8,000 words. The number of tokens processed in an input grows as more complex LLMs are trained.

- **Transformer**

It is a neural network architecture that has served as the basis for deep learning models used in modern generative AI models. Transformers specialize in processing sequential information, such as natural language, and for this, they employ the self-attention mechanism, where different weights are given to each part of the input. These models take a sentence in the form of a sequence of vectors, convert it into an encoded vector, which is then decoded into an output sequence. This architecture was proposed by a group of Google researchers in 2017, in the seminal paper "Attention is all you need."

- **RAG**

Retrieval-Augmented Generation (RAG) is a methodology to optimize LLMs so that they take into account a reference knowledge base when generating responses. This way, with RAG, we augment LLMs' capabilities with internal or specific information that has been trained with large volumes of generic data.